

Projet SUPMeal



Documentation Technique

Par Julien RENOULT

Table des matières

Introduction	5
Technologies utilisées.....	6
PostgreSQL.....	6
Python	6
Django	6
Basedpyright/Ruff.....	7
Autres librairies.....	7
Nuxt/VueJs	8
Linters.....	8
Tiptap.....	8
Autres librairies.....	9
Déploiement du projet	10
Prérequis au lancement du projet.....	10
Lancement de l'application	11
Lancer les migrations Django	12
Lancer les tests/linters backend et frontend (uniquement dev)	12
Architecture du projet	13
Vue d'ensemble.....	13
Services	13
Comment les services communiquent.....	14
Cycle de vie d'une requête	15
Organisation des apps backend	15
Organisation du frontend	16
Dev vs prod.....	16
Charte graphique.....	18
Couleurs.....	18
Typographie	18
Boutons	18
Icônes.....	18
Autres éléments	19
Backend.....	20
Base de données	20
Vue d'ensemble.....	20
Tables SQL.....	21
Modèles Django	30

Exécuter les migrations	32
API	34
Authentification	34
Pagination	35
Format des erreurs.....	35
Documentation interactive.....	36
1. Comptes & authentification (<code>users</code>).....	36
2. Cookbooks (<code>cookbooks</code>)	49
3. Recettes, tags & ingrédients (<code>recipes</code>).....	58
4. Planning (<code>planning</code>).....	66
5. Messagerie (<code>messaging</code>).....	70
Oauth.....	76
Lier Microsoft à un compte existant	78
Variables d'environnement.....	79
Frontend	81
Vue d'ensemble.....	81
Composants	82
Branding & boutons.....	82
Commun	83
Cookbook	83
Formulaires	85
Import / export	86
Layout.....	86
Planning.....	87
Recettes	88
Recherche	91
Paramètres.....	92
Icônes	92
Stores.....	93
<code>useCookbookStore</code>	93
<code>useMessageStore</code> (<code>vide</code>).....	94
<code>useImportExportStore</code>	94
<code>usePlanningStore</code>	94
<code>useRecipeStore</code>	95
<code>useToastStore</code>	96
<code>useUserStore</code>	96
Composables	98
Pages principales.....	102

Auth / public.....	103
Recettes	104
Cookbooks	105
Planning.....	106
Accueil, recherche, paramètres & divers	107
Dépôt Git.....	108

Introduction

Cette documentation technique a pour objectif de présenter l'architecture de notre projet ainsi que les étapes nécessaires pour le lancer en environnement local. Ce projet consiste en la création d'une application web de gestion de recettes et de livres de recettes (cookbooks).

Afin de présenter l'ensemble du projet de manière claire et structurée, cette documentation sera organisée en plusieurs parties, présentées de la manière suivante :

- [Technologies utilisées](#)
- [Déploiement du projet](#)
- [Architecture du projet](#)
- [Charte graphique](#)
- [Backend](#)
- [Frontend](#)
- [Dépôt Git](#)

Technologies utilisées

Dans cette partie, nous allons présenter les différentes technologies utilisées dans le cadre de cette application web. Nous aborderons tout d'abord **PostgreSQL** pour la gestion de la base de données, puis **Python** et ses différentes librairies pour le backend. Enfin, nous terminerons par **Nuxt** et ses différentes librairies pour la mise en place de la vue web de l'application.

PostgreSQL

Pour gérer à la fois une potentielle grande quantité de données et des relations complexes entre les différentes entités, il était nécessaire de choisir un **SGBD** capable de supporter ces contraintes. Nous nous sommes donc orientés vers **PostgreSQL**, qui répond à ces différents besoins.

En effet, ce SGBD permet de gérer efficacement d'importants volumes de données et offre une excellente prise en charge des relations complexes entre les différentes tables. Il permet de répondre à nos deux contraintes pour une application de gestion de recettes de cuisines robustes.



Python

Ayant des compétences beaucoup plus orientés vers **Python**, j'ai décidé de me rabattre sur ce langage. Ce langage à la fois simple et robuste par son typage nous permet de notamment offrir un code de beaucoup plus grande qualité. De plus, grâce à ses très nombreuses librairies et frameworks backend proposés, il nous permet d'en proposer un service approprié à cette application web. Je vais maintenant présenter ces différentes librairies utilisées avec ce langage.



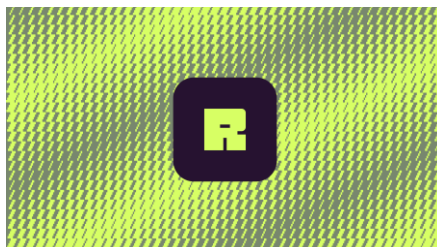
Django

Django est un framework web Python robuste et mature, idéal pour développer rapidement des applications web robuste et de qualité. L'utilisation principale de ce framework est qu'il intègre de nombreuses fonctionnalités prêtes à l'emploi (ORM, admin, authentification, etc). De plus, l'extension **REST framework** ajoute une couche d'API REST puissante et flexible au-dessus de Django ce qui le rends le framework idéal pour la création de l'API pour l'application web.



Basedpyright/Ruff

Pour contrôler la qualité du code écrite, deux librairies sortent du lot. L'un permet de vérifier les types statiques pour Python afin de détecter des erreurs à la compilation (**basedpyright**). Il améliore la stabilité et la maintenabilité du code. De l'autre, **Ruff** est un linter rapide qui détecte les erreurs de style et du codage. Il aide à maintenir un code propre et consistant.



Autres librairies

Librairie	Description
django-filter	Filtrage de querysets basé sur les paramètres de requêtes
django-cors-headers	Gestion des headers CORS pour les requêtes cross-origin
django-allauth	Intégration de providers d'authentification tiers (OAuth)
dj-rest-auth	Endpoints d'authentification pour Django Rest Framework
djangoestframework-simplejwt	Gestion de tokens d'authentification JSON Web Tokens
drf-spectacular	Documentation automatique d'API via OpenAPI/Swagger
django-envIRON	Gestion des paramètres de configuration via variables d'environnement
psycopg[binary]	Connecteur base de données PostgreSQL
gunicorn	Serveur WSGI production-ready pour Python
whitenoise	Serving de fichiers statiques pour applications Django

Nuxt/VueJs

Concernant le développement du frontend, il a été privilégié de choisir **Vue.js**. Ce choix s'explique notamment par la simplicité qu'offre **Vue.js** pour séparer la partie visuelle (HTML, CSS, etc.) de la logique applicative et fonctionnelle (**TypeScript**). De plus, ayant déjà de l'expérience dans ce framework, cela facilite grandement le développement du visuel web.

Pour le framework frontend, nous avons choisi **Nuxt.js**, particulièrement adapté au développement d'applications web modernes et évolutives. Grâce à son organisation structurée en différents éléments (pages, composants, composables, etc.) et au respect du principe de "Single Responsibility", **Nuxt** permet de distinguer clairement la couche d'interface utilisateur de la logique métier et des appels à l'API backend avec notamment les « Use » et les « Stores ».

Cette architecture facilite la maintenance, l'évolution du projet et la réutilisation du code tout en garantissant une structure cohérente et bien organisée. **Nuxt** constitue donc un choix particulièrement pertinent pour le développement de notre application web.



Linters

Pour vérifier la qualité du code écrite dans le framework, trois extensions vont être utilisées. D'abord, nous avons l'intégration de l'**ESLint** qui est un linter pour vérifier le TypeScript. Ensuite, nous avons installé **prettier** qui est un outil de formatage de code multi-langages très utile dans notre framework front-end. Enfin, **stylelint** va être utilisé pour le CSS spécifiquement.



Tiptap

Afin de proposer un éditeur markdown assez poussé pour l'édition des recettes, nous avons choisi la librairie **tiptap**. Il en vient sous trois extensions permettant d'aider dans la gestion de formatage de texte en markdown :

- **@tiptap/vue-3** : intégration de l'éditeur Tiptap pour Vue 3
- **@tiptap/starter-kit** : plugins de base pour Tiptap (gras, italique, titres...)
- **@tiptap/extension-placeholder** : extension pour ajouter un placeholder



Autres librairies

Librairie	Description
@nuxtjs/tailwindcss	Intégration de Tailwind CSS dans Nuxt
@pinia/nuxt	Intégration du gestionnaire d'état Pinia dans Nuxt
@vueuse/nuxt	Collection de composables Vue pour Nuxt
openapi-typescript	Génération de types TypeScript depuis une spec OpenAPI
vitest	Framework de test unitaire pour Vite
zod	Validation de schémas et parsing déclaratif

Déploiement du projet

Dans cette partie, nous vous présenterons comment déployer ce projet. On abordera d'abord les prérequis, ensuite comment lancer le projet avec docker compose. Enfin, nous aborderons les différentes commandes essentielles à connaître pour lancer les tests et les linters pour d'une part vérifier les fonctionnalités du backend et de l'autre la qualité du code écrite dans le backend et frontend.

Prérequis au lancement du projet

Avant de lancer le projet, assurez-vous d'avoir installé les outils suivants :

- Docker ([Docker Desktop](#))
- Docker-compose

Si vous avez au moins installé Docker et Docker compose, vous pouvez passer à la suite.

Mettez un fichier `.env` à la **racine du projet** en remplaçant les valeurs d'environnements par ceux que vous voulez :

```
DEBUG=True

SECRET_KEY=ChangeMe

ALLOWED_HOSTS=localhost,127.0.0.1

DATABASE_NAME=recipes
DATABASE_USER=recipes
DATABASE_PASSWORD=recipes
DATABASE_HOST=postgres
DATABASE_PORT=5432
POSTGRES_PORT=5436

BACKEND_PORT=8008

FRONTEND_PORT=3008

# Nuxt's runtimeConfig.public.apiUrl (nuxt.config.ts) - NUXT_PUBLIC_API_URL is
# Nuxt's standard env override name for that key (NUXT_PUBLIC_ +
# SCREAMING_SNAKE_CASE).
# Read both server-side (SSR) and client-side (browser fetch), so in prod
# this must be the API's real public URL, not an internal Docker service name.
NUXT_PUBLIC_API_URL=http://localhost:8008/api

CORS_ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3008

AZURE_CLIENT_ID='azure_client_id'
AZURE_TENANT_ID='azure_tenant_id'
AZURE_CLIENT_SECRET='azure_client_secret'
AZURE_REDIRECT_URI='http://localhost:3008/connect/microsoft/callback'
AZURE_AUTHORITY='https://login.microsoftonline.com/common/'
```

Mettez dans le **.env.local** du **frontend** cette valeur :

```
NEXT_PUBLIC_API_URL=http://localhost:8008/api
```

Mettez dans le fichier **.env.local** du **backend** remplaçant les valeurs secret pour l'OAuth présentes ci-dessous :

```
AZURE_CLIENT_ID='azure_client_id'  
AZURE_TENANT_ID='azure_tenant_id'  
AZURE_CLIENT_SECRET='azure_client_secret'  
AZURE_REDIRECT_URI='http://localhost:3008/connect/microsoft/callback'  
AZURE_AUTHORITY='https://login.microsoftonline.com/common/'
```

Lancement de l'application

Pour lancer l'application en mode développement ou en production, vous pouvez lancer une de ces deux commandes :

```
# Version dev  
  
docker compose -f docker-compose.dev.yml up --build  
  
# Version prod  
docker compose -f docker-compose.prod.yml up --build
```

Pour l'arrêter via le terminal, lancer une de ces deux commandes :

```
# Avec suppression des volumes  
docker compose -f docker-compose.dev.yml down -v  
  
# Sans suppression des volumes  
docker compose -f docker-compose.dev.yml down  
  
# Avec suppression des volumes  
docker compose -f docker-compose.prod.yml down -v  
  
# Sans suppression des volumes  
docker compose -f docker-compose.prod.yml down
```

Si vous lancez pour la première fois l'application, veuillez attendre quelques minutes le temps que tout le build se fasse avec les migrations Django.

Lancer les migrations Django

Pour appliquer les migrations dans le conteneur Django :

```
docker-compose -f docker-compose.dev.yml exec backend uv run python manage.py makemigrations

docker-compose -f docker-compose.dev.yml exec backend uv run python manage.py migrate

docker-compose -f docker-compose.prod.yml exec backend python manage.py migrate
```

`exec backend` permet d'exécuter les commandes dans le conteneur du service `backend`.

Lancer les tests/linters backend et frontend (uniquement dev)

Pour lancer les tests de **pytest**, vous pouvez lancer un de cette commande :

```
docker-compose -f docker-compose.dev.yml exec backend uv run pytest
```

Pour lancer les linters **Python**, vous pouvez lancer ces deux commandes :

```
docker-compose -f docker-compose.dev.yml exec backend uv run ruff check .

docker-compose -f docker-compose.dev.yml exec backend uv run basedpyright
```

Pour lancer les linters côté **Nuxt**, vous pouvez lancer ces différentes commandes :

```
# Pour lancer les linters

docker-compose -f docker-compose.dev.yml exec frontend npm run lint
docker-compose -f docker-compose.dev.yml exec frontend npm run typecheck

# Pour formater les fichiers
docker-compose -f docker-compose.dev.yml exec frontend npm run format
```

Architecture du projet

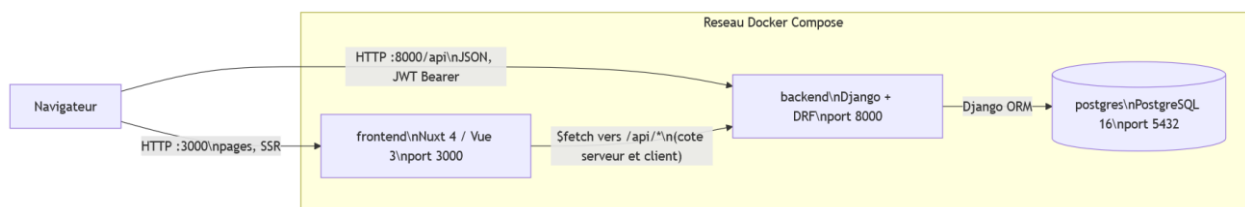
Vue d'ensemble

SUPMEAL est une application web de recettes / cookbooks / planification de repas, construite comme trois conteneurs orchestrés par Docker Compose :

- **postgres** - PostgreSQL 16, la source de vérité unique.
- **backend** - Django + Django REST Framework, expose une API JSON sous `/api/`.
- **frontend** - Nuxt 4 (Vue 3), une SPA rendue côté serveur qui consomme cette API en HTTP.

Il n'y a pas de couche temps réel/websocket séparée : tout, y compris la fonctionnalité de discussion par cookbook, passe par de simples appels REST (voir « Comment les services communiquent »).

Services



- Le **navigateur** parle aux deux services **frontend** (pour les pages/SSR) et directement à **backend** (pour les appels API faits côté client, puisque `useApi()` cible `runtimeConfig.public.apiUrl` – voir [nuxt.config.ts](#)).
- **frontend** peut aussi appeler **backend** côté serveur pendant le SSR, avec la même `apiUrl`.
- **backend** est le seul service autorisé à joindre **postgres**. En prod (`docker-compose.prod.yml`), **postgres** ne publie aucun port sur l'hôte, uniquement le réseau interne compose – voir « Dev vs prod ».
- Rien d'autre ne se trouve sur le chemin de la requête : pas de reverse proxy, pas de file de messages, pas de cache, pas de serveur websocket. L'authentification est du JWT sans état, donc n'importe quelle instance backend peut servir n'importe quelle requête.

Comment les services communiquent

Frontend → Backend : REST en JSON, auth JWT bearer.

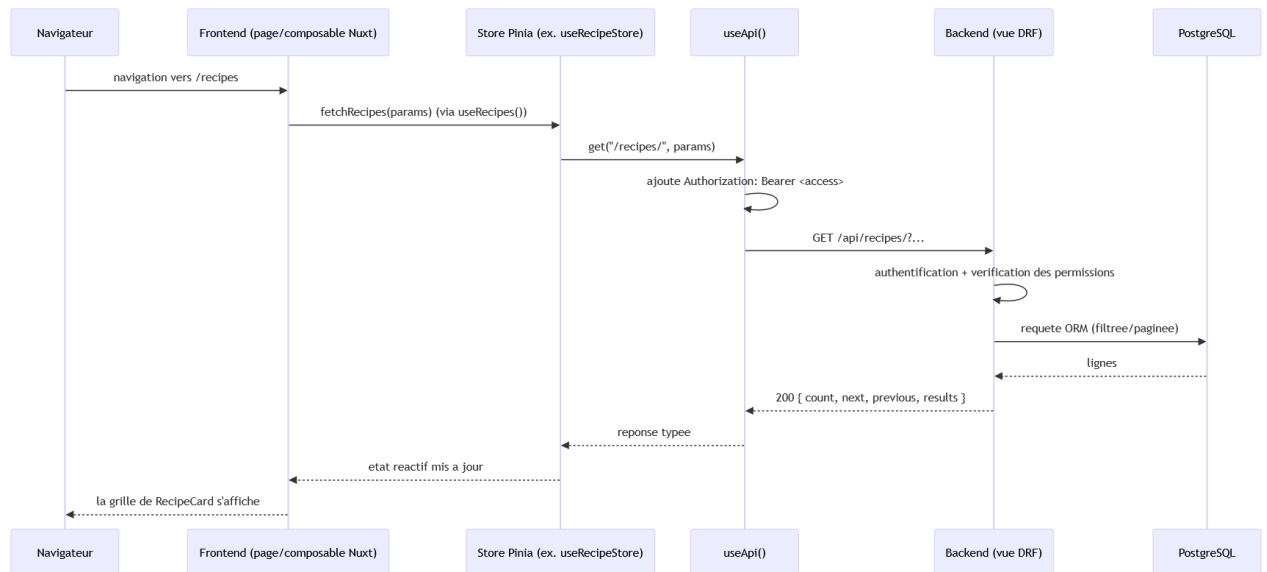
- `frontend/app/composables/useAPI.ts` (`useApi()`) est le client http unique utilisé partout dans le frontend : `get/post/put/patch/del` construit sur `$fetch` de Nuxt, URL de base issue de `runtimeConfig.public.apiUrl`.
- Il lit le token d'accès depuis `useToken()` et pose `Authorization: Bearer <access>` sur chaque requête ; `Content-Type` est mis à `application/json` sauf si le corps est un `FormData` (upload d'image/fichier).
- Sur un 401, il vide la session locale et redirige vers `/login` côté client - il n'y a pas de rejeu silencieux via le refresh token à l'intérieur de `useApi()` lui-même (le refresh est un appel explicite séparé).
- Aucun composable ni composant n'appelle `$fetch/fetch` directement vers le backend en dehors de `useApi()`, à une exception volontaire près : `CookbookDiscussionSidebar.vue` utilise lui aussi `useApi()` directement (pas via un store Pinia) pour appeler les routes de messagerie imbriquées (`/cookbooks/{id}/messages/`, `/cookbooks/{id}/recipes/{id}/messages/`, `/cookbooks/{id}/plannings/{id}/messages/`)
- Tout le reste passe par un store Pinia (`useRecipeStore`, `useCookbookStore`, `usePlanningStore`, `useUserStore`, `useImportExportStore`) qui encapsule des appels `useApi()`

Backend → Base de données : Django ORM, synchrone, WSGI.

- `backend/config/wsgi.py` est ce qui sert réellement les requêtes (`gunicorn config.wsgi:application` en prod, `runserver` en dev, voir Dev vs prod). `asgi.py` existe (scaffolding par défaut de Django) mais rien dans la stack ne l'exécute - il n'y a pas de serveur Channels/ASGI dans `pyproject.toml`, donc pas de support websocket malgré la présence du fichier.
- L'authentification utilise `djangorestframework-simplejwt` : access token 60 min, refresh token 7 jours. `POST /api/users/logout/` blackliste les deux.
- L'OAuth (Microsoft uniquement, pour l'instant) est géré **entièrement côté serveur** - le frontend ne voit jamais le secret client, il ne fait que transmettre un `code` d'autorisation au backend.

Cycle de vie d'une requête

Lecture authentifiée typique (ex : ouvrir la liste des recettes), de bout en bout :



Si le token d'accès est manquant/expiré, `useApi()` réagit au 401 qui en résulte en vidant la session et en redirigeant vers `/login` (voir Comment les services communiquent) ; le rafraîchir avant que cela n'arrive est un flux explicite séparé.

Organisation des apps backend

Le backend est découpé en cinq apps Django par domaine, plus un petit module `common` pour les aides transverses (aujourd'hui uniquement la validation d'upload d'image, `backend/common/image_validation.py`).

App	Responsabilité
<code>users</code>	Comptes, auth JWT, OAuth Microsoft
<code>cookbooks</code>	Cookbooks + partage
<code>recipes</code>	Recettes, tags, ingrédients
<code>planning</code>	Planification de repas
<code>messaging</code>	Fils de discussion par cookbook/recette/planning

`config/` est le projet Django lui-même : `settings.py`, `urls.py` (monte le routeur de chaque app sous `/api/`), `wsgi.py`/`asgi.py`. `drf-spectacular` sert une UI OpenAPI interactive sur `/api/docs/`.

Organisation du frontend

Le frontend est une app Nuxt 4 standard (`frontend/app/`), organisée par nature de fichier plutôt que par fonctionnalité :

Dossier	Contenu
<code>pages/</code>	Routes basées sur les fichiers - une page par écran
<code>components/</code>	Composants Vue présentationnels/réutilisables, groupés par sous-dossier de domaine
<code>composables/</code>	Logique réactive réutilisable (<code>use*.ts</code>), y compris les composables « edit view » qui relie une page à son store
<code>stores/</code>	Stores Pinia - le seul endroit (à part <code>CookbookDiscussionSidebar.vue</code> , voir ci-dessus) qui appelle <code>useApi()</code>
<code>layouts/</code>	<code>app.vue</code> (coquille authentifiée : sidebar + toasts + sidebar de discussion) et <code>empty.vue</code> (nue)
<code>middleware/</code>	<code>auth.global.ts</code> - garde de route exécutée à chaque navigation, protège les routes authentifiées et redirige les utilisateurs déjà connectés hors des pages réservées aux invités

Dev vs prod

Les deux sont de simples Docker Compose, sans orchestrateur, définis côte à côte à la racine du dépôt :

	<code>docker-compose.dev.yml</code>	<code>docker-compose.prod.yml</code>
Serveur backend	<code>manage.py runserver</code> (autoreload), dépendances installées via <code>uv sync</code> au démarrage du conteneur	<code>gunicorn config.wsgi:application --workers 3</code> , dépendances intégrées à l'image au build
Serveur frontend	<code>npm run dev -- --host</code> (HMR)	Image construite depuis <code>frontend/Dockerfile</code> (build Nuxt de production)

	<code>docker-compose.dev.yml</code>	<code>docker-compose.prod.yml</code>
Code source	Monté en bind (<code>./backend:/app</code> , <code>./frontend:/app</code>) pour l'édition à chaud	Non monté - le code est intégré à l'image
Port Postgres	Publié sur l'hôte (<code>\${POSTGRES_PORT}:5432</code>), pratique pour un client DB local	Non publié - joignable uniquement depuis <code>backend</code> via le réseau compose
Fichiers statiques	Servis par <code>runserver</code>	<code>collectstatic</code> s'exécute avant le démarrage de <code>gunicorn</code>

Les deux partagent le même `.env` (via `env_file:`) pour `DATABASE_*`, `BACKEND_PORT`, `FRONTEND_PORT`, `AZURE_*`, etc.

Charte graphique

Couleurs

Couleur	Code Hexadécimal
Vert Clair	#64FA16
Vert Foncé	#1A7F02
Gris Clair	#F5F5F5
Gris Foncé	#333333
Rouge (Erreur)	#E74C3C
Jaune (Warning)	#F1C40F
Blanc	#FFFFFF
Gris (Bordures)	#E0E0E0

Typographie

- Police principale : Inter
- Titre 2 (h2) : Inter Semibold - Bold / Semibold
- Titre 3 (h3) : Inter / SemiBold
- Texte : Inter Regular

Boutons

- Bouton principal : Vert Foncé (#1A7F02) - Texte Blanc
- Bouton secondaire : Gris Clair (#F5F5F5) - Texte Gris Foncé (#333333)
- Bouton destructif : Rouge (#E74C3C) - Texte Blanc

Icônes

- Icônes de la même couleur que le texte associé
- Possibilité d'utiliser des icônes de couleur pour mettre en avant certains éléments (ex: icône verte pour validation)

Autres éléments

- Carte (Card) : Fond Blanc - Bordure Gris Clair (#E0E0E0)
- Champ de saisie (Input) : Fond Blanc - Bordure Gris (#E0E0E0) - Texte Gris Foncé (#333333)
- Barre de progression (ProgressBar) : Fond Gris Clair (#F5F5F5) - Barre de progression Vert Clair (#64FA16)

Cette charte graphique définit les couleurs principales, la typographie et le style des principaux composants de l'interface de **SUPMEAL**. Elle assure une cohérence visuelle et une bonne lisibilité à travers toute l'application. Les couleurs choisies reflètent l'identité de marque de **SUPMEAL** avec des tons verts pour évoquer la nature et la fraîcheur. Le contraste entre les différents gris et le blanc assure une bonne hiérarchie visuelle. La police Inter, disponible sur Google Fonts, offre une excellente lisibilité sur écran et une personnalité moderne et friendly.

Backend

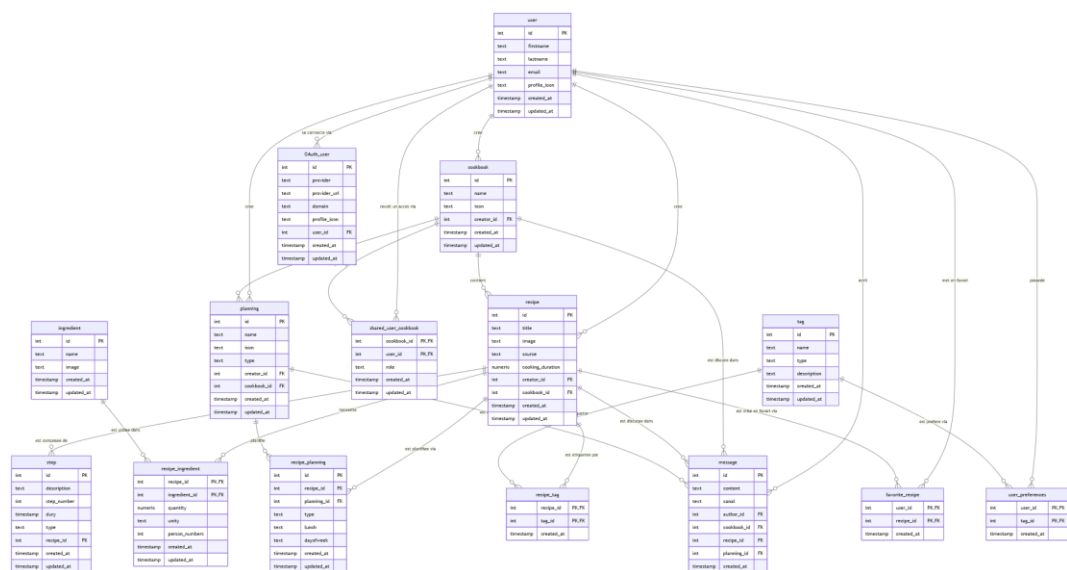
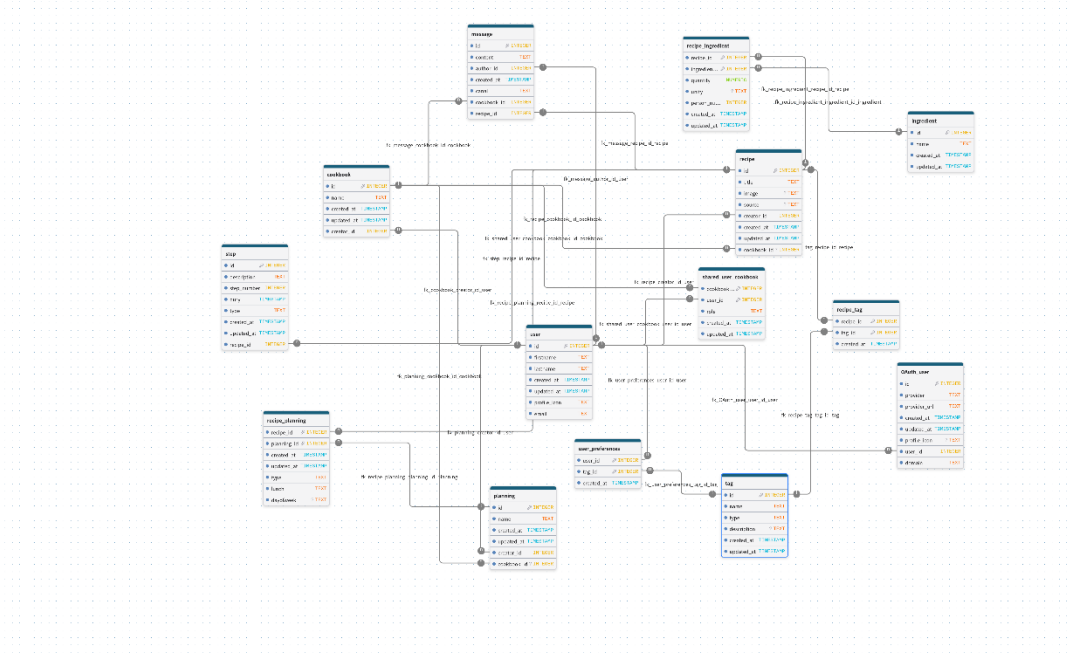
Ici, nous allons présenter la base de données et l'API créés pour l'application web avec des explications détaillées sur chaque attribut et appel.

Base de données

Ici, nous allons présenter la base de données créée pour l'application web.

Vue d'ensemble

- **Système de base de données** : PostgreSQL 16 (voir `docker-compose.dev.yml`, service `postgres`)
- **Domaine** : un gestionnaire de recettes/carnets de cuisine – les utilisateurs rédigent des recettes, les organisent en cookbooks partagés, planifient des repas et discutent autour de ceux-ci.



Tables SQL

`user`

Le compte d'une personne utilisant l'application. Toutes les autres tables ayant une notion de « créateur », « auteur » ou « propriétaire » pointent vers cette table.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>firstname</code>	TEXT	not null	
<code>lastname</code>	TEXT	not null	
<code>email</code>	TEXT	not null	
<code>password</code>	TEXT	not null	mot de passe haché (fourni par l' <code>AbstractUser</code> de Django)
<code>profile_icon</code>	TEXT	not null, default ''	optionnel ; chaîne vide si non fourni
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`OAuth_user`

Un fournisseur d'identité externe (Google, GitHub, ...) lié à un `user`, de sorte qu'un même compte puisse se connecter via plusieurs fournisseurs.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>provider</code>	TEXT	not null	ex. "google", "github"
<code>provider_url</code>	TEXT	not null	
<code>domain</code>	TEXT	not null	
<code>profile_icon</code>	TEXT	null	

Colonne	Type	Contraintes	Note
<code>user_id</code>	INTEGER	not null	→ <code>user.id</code>
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`cookbook`

Une collection nommée de recettes/planning appartenant à un utilisateur, éventuellement partagée avec d'autres.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>name</code>	TEXT	not null	
<code>icon</code>	TEXT	null, défaut : icône intégrée	data URI/base64 ; ne fait pas partie du schéma d'origine, ajoutée par-dessus uniquement pour l'affichage
<code>creator_id</code>	INTEGER	not null	→ <code>user.id</code>
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`shared_user_cookbook`

Accorde à un utilisateur l'accès à un cookbook qu'il n'a pas créé, avec un `role` (niveau de permission). Clé primaire composite : un utilisateur ne peut recevoir l'accès à un cookbook donné qu'une seule fois.

Colonne	Type	Contraintes	Note
<code>cookbook_id</code>	INTEGER	PK	→ <code>cookbook.id</code>
<code>user_id</code>	INTEGER	PK	→ <code>user.id</code>
<code>role</code>	TEXT	not null	
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`recipe`

Une recette : titre, source/image optionnelles, et un créateur. `cookbook_id` est nullable, car une recette peut exister avant d'être rangée dans un cookbook.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>title</code>	TEXT	not null	
<code>image</code>	TEXT	null	
<code>source</code>	TEXT	null	
<code>cooking_duration</code>	NUMERIC	null	ex. en minutes
<code>creator_id</code>	INTEGER	not null	→ <code>user.id</code>
<code>cookbook_id</code>	INTEGER	null	→ <code>cookbook.id</code>
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`step`

Une étape ordonnée des instructions d'une recette.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>description</code>	TEXT	not null	
<code>step_number</code>	INTEGER	not null	ordre au sein de la recette
<code>dury</code>	TIMESTAMP	not null	durée/horaire de l'étape
<code>type</code>	TEXT	not null	ex. "prep", "cook"
<code>recipe_id</code>	INTEGER	not null	→ <code>recipe.id</code>
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`ingredient`

Un ingrédient réutilisable (ex. « farine », « œuf ») partagé entre les recettes. La quantité utilisée dans une recette précise n'est pas stockée ici.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>name</code>	TEXT	not null	
<code>image</code>	TEXT	null	
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`recipe_ingredient`

Table de jointure quantifiant la quantité d'un ingrédient nécessaire à une recette. Clé primaire composite : une recette ne liste un ingrédient donné qu'une seule fois.

Colonne	Type	Contraintes	Note
<code>recipe_id</code>	INTEGER	PK	→ <code>recipe.id</code>
<code>ingredient_id</code>	INTEGER	PK	→ <code>ingredient.id</code>
<code>quantity</code>	NUMERIC	not null	
<code>unity</code>	TEXT	Null	ex. "g", "mL"
<code>person_numbers</code>	INTEGER	not null	quantité prévue pour N personnes
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`tag`

Une étiquette servant à catégoriser les recettes (régime, cuisine, difficulté, ...) ou à exprimer les préférences gustatives d'un utilisateur.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>name</code>	TEXT	not null	
<code>type</code>	TEXT	not null	
<code>description</code>	TEXT	null	
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`recipe_tag`

Table de jointure attachant un `tag` à une `recipe`. Clé primaire composite : une recette ne peut pas porter deux fois la même étiquette.

Colonne	Type	Contraintes	Note
<code>recipe_id</code>	INTEGER	PK	→ <code>recipe.id</code>
<code>tag_id</code>	INTEGER	PK	→ <code>tag.id</code>
<code>created_at</code>	TIMESTAMP	not null	

`user_preferences`

Table de jointure enregistrant qu'un utilisateur apprécie/suit une `tag` donnée (utilisé pour personnaliser les recommandations). Clé primaire composite.

Colonne	Type	Contraintes	Note
<code>user_id</code>	INTEGER	PK	→ <code>user.id</code>
<code>tag_id</code>	INTEGER	PK	→ <code>tag.id</code>
<code>created_at</code>	TIMESTAMP	not null	

`favorite_recipe`

Indique qu'un utilisateur a mis une recette en favori, pour un affichage personnalisé des recettes (ex. filtre « mes favoris »). Ne fait pas partie du schéma SQL d'origine - ajoutée par-dessus, sur le même modèle qu'`user_preferences` (clé primaire composite, pas de `updated_at`).

Colonne	Type	Contraintes	Note
<code>user_id</code>	INTEGER	PK	→ <code>user.id</code>
<code>recipe_id</code>	INTEGER	PK	→ <code>recipe.id</code>
<code>created_at</code>	TIMESTAMP	not null	

planning

Un plan de repas nommé, créé par un utilisateur, éventuellement rattaché à un cookbook. `type` et `icon` ne font pas partie du schéma SQL d'origine, ajoutés par-dessus : `type` distingue un plan sur une seule journée (`journalier`, jusqu'à 6 repas - un créneau `lunch` × `type` par jour) d'un plan couvrant une semaine complète (`hebdomadaire`, jusqu'à 42 repas), et `icon` reprend le même principe que `cookbook.icon` (affichage uniquement, icône intégrée par défaut).

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>name</code>	TEXT	not null	
<code>icon</code>	TEXT	null, défaut : icône intégrée	data URI/base64
<code>type</code>	TEXT	not null, défaut 'hebdomadaire'	<code>journalier</code> ou <code>hebdomadaire</code>
<code>creator_id</code>	INTEGER	not null	→ <code>user.id</code>
<code>cookbook_id</code>	INTEGER	null	→ <code>cookbook.id</code>
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

`recipe_planning`

Planifie une `recipe` au sein d'un `planning`, pour un créneau jour/repas/plat donné. Contrairement aux autres tables de jointure de ce schéma, elle utilise un `id` de substitution plutôt qu'une clé primaire composite : la même recette peut être planifiée plusieurs fois dans un planning (ex. le même dessert plusieurs jours), donc `(recipe_id, planning_id)` ne peut pas être la clé à elle seule.

À la place, une contrainte d'unicité sur `(planning_id, dayofweek, lunch, type)` garantit au plus une recette par créneau jour/moment-du-repas/plat.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>recipe_id</code>	INTEGER	not null	→ <code>recipe.id</code>
<code>planning_id</code>	INTEGER	not null	→ <code>planning.id</code>
<code>type</code>	TEXT	not null	
<code>lunch</code>	TEXT	not null	
<code>dayofweek</code>	TEXT	null	
<code>created_at</code>	TIMESTAMP	not null	
<code>updated_at</code>	TIMESTAMP	not null	

message

Un message posté par un utilisateur, toujours rattaché à un cookbook. `recipe_id` et `planning_id` sont tous deux nullables **et mutuellement exclusifs** (imposé par une contrainte `CHECK`, `message_not_both_recipe_and_planning`) : un message sans aucun des deux appartient au canal global du cookbook, un message avec `recipe_id` défini appartient au canal de cette recette précise, et un message avec `planning_id` défini appartient au canal de ce planning précis (la recette/le planning doit lui-même être rangé dans `cookbook_id`). Les messages n'ont pas de `updated_at` - ils ne peuvent être que postés ou supprimés, jamais modifiés.

Colonne	Type	Contraintes	Note
<code>id</code>	INTEGER	PK	
<code>content</code>	TEXT	not null	
<code>canal</code>	TEXT	not null	canal de conversation
<code>author_id</code>	INTEGER	not null	→ <code>user.id</code>
<code>cookbook_id</code>	INTEGER	not null	→ <code>cookbook.id</code>
<code>recipe_id</code>	INTEGER	null	→ <code>recipe.id</code> ; mutuellement exclusif avec <code>planning_id</code>
<code>planning_id</code>	INTEGER	null	→ <code>planning.id</code> ; mutuellement exclusif avec <code>recipe_id</code>
<code>created_at</code>	TIMESTAMP	not null	

Toutes les clés étrangères sont déclarées `ON UPDATE NO ACTION ON DELETE NO ACTION` dans le schéma d'origine : une ligne référencée ne peut pas être supprimée tant que d'autres lignes pointent encore vers elle. Voir la partie suivante pour la façon dont cela se traduit dans le comportement `on_delete` de Django.

Modèles Django

Organisation des apps

Le schéma ci-dessus est implémenté sous forme de cinq apps Django dans `backend/`, découpées par domaine plutôt qu'en une seule grosse app :

App	Responsabilité
<code>users</code>	Comptes (<code>User</code> , l' <code>AUTH_USER_MODEL</code> du projet) et identités OAuth (<code>OAuthUser</code>)
<code>cookbooks</code>	Cookbook et le partage de cookbooks (<code>SharedUserCookbook</code>)
<code>recipes</code>	Recipe et tout ce qui s'y rattache : <code>Ingredient</code> , <code>Tag</code> , <code>Step</code> , plus les tables de jointure <code>RecipeIngredient</code> / <code>RecipeTag</code> / <code>UserPreference</code> / <code>FavoriteRecipe</code>
<code>planning</code>	Planning et <code>RecipePlanning</code>
<code>messaging</code>	Message

Ordre des dépendances (qui importe/référence qui) : `users` → `cookbooks` → `recipes` → `planning` → `messaging` (`messaging.models` importe à la fois `recipes.models.Recipe` et `planning.models.Planning`, puisqu'un message peut être rattaché à l'un ou l'autre). `users` elle-même ne dépend de rien d'autre dans le projet - cela a de l'importance pour les migrations, voir plus bas.

Correspondance table ↔ modèle

Table SQL	App Django	Modèle	Table réelle en base
<code>user</code>	<code>users</code>	<code>User</code>	<code>users_user</code>
<code>OAuth_user</code>	<code>users</code>	<code>OAuthUser</code>	<code>users_oauthuser</code>
<code>cookbook</code>	<code>cookbooks</code>	<code>Cookbook</code>	<code>cookbooks_cookbook</code>
<code>shared_user_cookbook</code>	<code>cookbooks</code>	<code>SharedUserCookbook</code>	<code>cookbooks_sharedusercookbook</code>
<code>recipe</code>	<code>recipes</code>	<code>Recipe</code>	<code>recipes_recipe</code>
<code>ingredient</code>	<code>recipes</code>	<code>Ingredient</code>	<code>recipes_ingredient</code>

Table SQL	App Django	Modèle	Table réelle en base
recipe_ingredient	recipes	RecipeIngredient	recipes_recipeingredient
tag	recipes	Tag	recipes_tag
recipe_tag	recipes	RecipeTag	recipes_recipetag
user_preferences	recipes	UserPreference	recipes_userpreference
favorite_recipe	recipes	FavoriteRecipe	recipes_favoriterecipe
step	recipes	Step	recipes_step
planning	planning	Planning	planning_planning
recipe_planning	planning	RecipePlanning	planning_recipeplanning
message	messaging	Message	messaging_message

Django crée également ses propres tables de support (django_migrations, django_content_type, django_session, django_admin_log, auth_permission, auth_group, ainsi que users_user_groups et users_user_permissions pour le système de permissions intégré) – il s'agit d'infrastructure du framework, pas d'éléments du schéma d'origine.

Modèle utilisateur personnalisé

La table `user` du schéma n'a pas de colonne mot de passe et le projet utilise django-allauth / dj-rest-auth pour l'authentification, donc `users.User` étend `AbstractUser` de Django (qui fournit déjà `username`, `password`, les permissions, etc.) et ajoute `email` (unique), `profile_icon`, `created_at` et `updated_at`. Il est déclaré dans `config/settings.py` ainsi :

```
AUTH_USER_MODEL = "users.User"
```

Chaque modèle ayant une clé étrangère « créateur »/« auteur »/« utilisateur » pointe vers `settings.AUTH_USER_MODEL` plutôt que d'importer `User` directement, ce qui est la façon standard, en Django, de référencer le modèle utilisateur actif sans créer de cycle d'import.

Toutes les clés étrangères utilisent `on_delete=models.PROTECT`, ce qui correspond au `ON DELETE NO ACTION` du schéma : supprimer une ligne encore référencée ailleurs lève une `ProtectedError` plutôt que de propager la suppression en cascade ou de mettre silencieusement la référence à `null`.

Clés primaires composites

La plupart des tables de jointure SQL (`recipe_tag`, `recipe_ingredient`, `shared_user_cookbook`, `user_preferences`, `favorite_recipe`) utilisent une clé primaire **composite** (ex. `(recipe_id, tag_id)`) plutôt qu'un `id` de substitution. Ceci est implémenté avec `models.CompositePrimaryKey`, natif de Django 6 et introduit spécifiquement pour ce cas d'usage.

Les arguments sont les *noms de champs* du modèle ("`recipe`", "`tag`"), pas les noms de colonnes en base (`recipe_id`, `tag_id`).

Une limitation : **l'admin Django ne peut pas encore enregistrer un modèle à clé primaire composite**. C'est pourquoi `RecipeTag`, `RecipeIngredient`, `SharedUserCookbook`, `UserPreference` et `FavoriteRecipe` ne sont pas enregistrés dans le `admin.py` de leur app (voir le commentaire en tête de chaque fichier). `RecipePlanning` fait exception puisqu'il utilise un `id` de substitution à la place.

Exécuter les migrations

Toutes les commandes ci-dessous s'exécutent via `docker-compose.dev.yml`, de sorte que Postgres et les bonnes versions de Python/Django soient utilisées - aucun environnement virtuel local n'est nécessaire. Démarrer d'abord la stack si elle ne tourne pas déjà :

```
docker compose -f docker-compose.dev.yml up -d postgres backend
```

Créer les migrations après toute modification d'un `models.py` (ceci inspecte les modèles et écrit les fichiers de migration sous `<app>/migrations/`) :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations
```

Appliquer les migrations à la base de données (ceci exécute le SQL sur Postgres) :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py migrate
```

En développement courant, le service `backend` exécute déjà `migrate` à chaque démarrage du conteneur (voir son `command` dans `docker-compose.dev.yml`), donc un simple `docker compose -f docker-compose.dev.yml up backend` suffit souvent après avoir récupéré de nouvelles migrations.

Vérifier les changements de modèles qui n'ont pas encore été transformés en migration (utile en CI, ou avant un commit) :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations --check --dry-run
```


Prévisualiser le SQL qu'une migration va exécuter, sans l'appliquer :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py sqlmigrate <app_label> <migration_number>
# ex. uv run python manage.py sqlmigrate recipes 0001
```

Revenir en arrière sur une app jusqu'à une migration antérieure :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py migrate <app_label>
<previous_migration_name>
# ex. uv run python manage.py migrate recipes 0001_initial
```

Exécuter le check système complet de Django (détecte les problèmes de configuration au-delà des simples migrations en attente) :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py check
```

Dépendances circulaires entre apps (piège des clés composites)

Quand plusieurs *nouvelles* apps avec des clés étrangères inter-apps sont migrées pour la première fois en un seul appel à `makemigrations`, l'autodétecteur de migrations de Django peut scinder la migration initiale de chaque app en deux : une migration `CreateModel` sans les champs relationnels, suivie d'une seconde migration qui ajoute (`AddField`) les clés étrangères une fois que les tables de toutes les apps existent. C'est la façon dont Django garantit un ordre de création valide entre apps, et c'est sans conséquence pour des clés étrangères normales.

Cela casse en revanche `CompositePrimaryKey` : la clé primaire a besoin que ses champs de clé étrangère constitutifs existent déjà sur le modèle au moment du `CreateModel`, puisque la contrainte `PRIMARY KEY` est émise dans le cadre de l'instruction `CREATE TABLE`. Si les champs FK sont reportés à une seconde migration, la création de la table échoue.

Solution de contournement : lors de la création de plusieurs nouvelles apps interdépendantes en même temps (ce qui était le cas ici), générer les migrations **une app à la fois, dans l'ordre des dépendances**, plutôt qu'un seul `makemigrations` global :

```
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations users
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations cookbooks
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations recipes
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations planning
docker compose -f docker-compose.dev.yml run --rm backend \
  uv run python manage.py makemigrations messaging
```

Chaque app est alors résolue par rapport aux apps dont les tables sont déjà définies, de sorte que chaque modèle - y compris les tables de jointure à clé composite - est créé en une seule migration.

Ceci ne concerne que l'**introduction de nouvelles apps** avec des relations inter-apps. Les modifications courantes des modèles d'une app existante (ajout d'un champ, ajustement d'un `Meta`, etc.) passent par un `makemigrations` normal, sans traitement particulier.

API

Cette partie liste chaque route API exposée par le backend, regroupée par app Django. Pour chaque route sont détaillés : les codes de statut de réponse attendus, ses paramètres (chemin, query et corps de requête), et le workflow attendu côté appelant.

Toutes les routes ci-dessous sont relatives à `/api/`. En local (via `docker-compose.dev.yml`), cela donne [http://localhost:\\${BACKEND_PORT}/api/](http://localhost:${BACKEND_PORT}/api/).

Authentification

L'API utilise des jetons JWT de type bearer (`django-rest-framework-simplejwt`). Une fois qu'un appelant dispose d'un token `access` (obtenu via inscription/connexion/OAuth), celui-ci doit être envoyé sur chaque requête authentifiée :

```
Authorization: Bearer <access>
```

- **Durée de vie de l'access token** : 60 minutes (`SIMPLE_JWT.ACCESS_TOKEN_LIFETIME`).
- **Durée de vie du refresh token** : 7 jours (`SIMPLE_JWT.REFRESH_TOKEN_LIFETIME`).
- Quand l'access token expire, il faut échanger le `refresh` token contre un nouveau via [POST /api/users/token/refresh/](#).
- [POST /api/users/logout/](#) blackliste à la fois le refresh token et l'access token ayant servi à l'appeler – voir `users.authentication.BlacklistAwareJWTAuthentication` - de sorte qu'aucun des deux ne peut être réutilisé ensuite, même si l'access token n'a pas encore expiré naturellement.
- Les routes sont soit `AllowAny` (aucun token requis), soit `IsAuthenticated` (n'importe quel access token valide, non blacklisté), soit soumises à une restriction supplémentaire (réservé au staff, réservé au propriétaire de l'objet, basé sur le rôle dans le cookbook) - chaque route ci-dessous précise laquelle s'applique.

Pagination

Les endpoints de liste qui gèrent la pagination (`DefaultPagination`, 10 éléments par page par défaut, 100 maximum via `?page_size=`) enveloppent leurs résultats dans ce format :

```
{
  "count": 42,
  "total_pages": 5,
  "current_page": 1,
  "next": "http://localhost:8000/api/recipes/?page=2",
  "previous": null,
  "results": [ /* ... */ ]
}
```

Tous les endpoints de liste ne sont pas paginés - `GET /api/tags/` et `GET /api/ingredients/` renvoient un simple tableau JSON, car aucune classe de pagination n'est configurée sur ces deux viewsets.

Format des erreurs

Les erreurs de validation (`400 Bad Request`) proviennent des serializers DRF, sous l'une de ces deux formes :

```
{ "detail": "Un message unique, lisible par un humain." }
```

ou une map d'erreurs par champ :

```
{ "field_name": ["This field is required."] }
```

`401 Unauthorized` est renvoyé quand l'authentification est absente, invalide ou expirée (entièrement géré par DRF/Simple JWT, pas par le code applicatif). `403 Forbidden` est renvoyé quand l'authentification a réussi mais que l'appelant n'a pas le rôle/la permission requise. `404 Not Found` est utilisé - volontairement, à plusieurs endroits ci-dessous - à la place de `403` quand révéler qu'un objet existe divulguerait une information que l'appelant ne devrait pas avoir (par exemple un cookbook dont il n'est pas membre).

Documentation interactive

Le schéma documenté manuellement ci-dessous est également généré automatiquement à partir du même code (via `drf-spectacular`) :

- Schéma OpenAPI brut : `GET /api/schema/`
- Interface Swagger : `GET /api/docs/`

1. Comptes & authentification (`users`)

`POST /api/users/register/`

Crée un nouveau compte local (username/email/mot de passe) et connecte immédiatement l'utilisateur.

Auth : `AllowAny`

Codes de statut

Statut	Signification
<code>201 Created</code>	Compte créé ; la réponse inclut l'utilisateur et une paire de tokens.
<code>400 Bad Request</code>	Champ manquant/invalid, username ou email déjà pris, ou mot de passe rejeté par les validateurs Django (trop court, trop courant, trop proche du username, entièrement numérique).

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>username</code>	string	oui	Unique.
Email	string	oui	Unique, doit être une adresse email valide.
<code>password</code>	string	oui	Écriture seule, validé par les validateurs de mot de passe Django.
<code>first_name</code>	string	non	
<code>last_name</code>	string	non	
<code>profile_icon</code>	string	non	Vaut "" par défaut si omis.

Workflow

1. Le client envoie le formulaire d'inscription en `POST`.
2. Le serializer valide l'unicité de `username/email` et la robustesse du mot de passe.
3. Le serveur hache le mot de passe (`set_password`) et crée la ligne `User`.
4. Le serveur émet une paire de JWT (`RefreshToken.for_user`), exactement comme une connexion classique.
5. Réponse : `{ "user": {...}, "access": "<jwt>", "refresh": "<jwt>" }`.
6. Le client stocke `access/refresh` et joint `Authorization: Bearer <access>` aux appels suivants.

`POST /api/users/login/`

Authentifie avec email + mot de passe.

Auth : `AllowAny`

Codes de statut

Statut	Signification
200 OK	Identifiants valides ; la réponse inclut l'utilisateur et une paire de tokens.
400 Bad Request	Email inconnu, mot de passe erroné, compte désactivé, ou compte OAuth-only sans mot de passe local - les trois cas renvoient le même message générique pour qu'un appelant ne puisse pas distinguer lequel s'applique.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
email	string	oui	
password	string	oui	Écriture seule.

Workflow

1. Le client envoie `{ email, password }` en `POST`.
2. Le serveur recherche le `User` par email, puis appelle `authenticate()` de Django avec le `username` de cet utilisateur et le mot de passe fourni.
3. En cas de succès, une nouvelle paire de JWT est émise (les tokens précédents, s'il y en a, ne sont **pas** révoqués par une nouvelle connexion).
4. Réponse : `{ "user": {...}, "access": "<jwt>", "refresh": "<jwt>" }`.

`POST /api/users/logout/`

Révoque la session courante : blackliste le refresh token fourni **ainsi que** l'access token ayant servi à authentifier cet appel.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Les deux tokens sont blacklistés. { "detail": "Logged out successfully." }
400 Bad Request	refresh manquant, malformé, expiré, déjà blacklisté, ou appartenant à un autre utilisateur que celui qui authentifie la requête.
401 Unauthorized	Access token absent/invalid/expiré/déjà blacklisté.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
refresh	String	oui	Le refresh token émis avec l'access token actuellement utilisé.

Workflow

1. Le client envoie `Authorization: Bearer <access>` et `{ "refresh": "<refresh>" }` dans le corps.
2. Le serveur parse `refresh` en tant que `RefreshToken` ; un échec de décodage/expiration/blacklist déjà en place renvoie immédiatement 400.
3. Le serveur vérifie que la claim `user_id` du refresh token correspond à `request.user.id` - sinon 400 (impossible de déconnecter la session de quelqu'un d'autre).
4. Le serveur blackliste le refresh token (`RefreshToken.blacklist()` - comportement standard de Simple JWT).
5. Le serveur blackliste également l'**access token** utilisé pour cette requête (`request.auth`), en créant manuellement les lignes `OutstandingToken/BlacklistedToken` que Simple JWT ne crée pas par défaut pour les access tokens.
6. À partir de ce moment : rafraîchir avec ce `refresh` token → 401 sur `/token/refresh/` ; réutiliser cet `access` token sur une route authentifiée quelconque → 401 (imposé par `BlacklistAwareJWTAuthentication`).

`POST /api/users/oauth/microsoft/`

Échange un `code` d'autorisation Microsoft contre un profil Graph, puis connecte l'utilisateur (création du compte à la première connexion). Voir pour le diagramme de séquence complet et les étapes d'intégration côté frontend.

Auth : `AllowAnonymous`

Codes de statut

Statut	Signification
<code>200 OK</code>	Connexion/inscription réussie ; la réponse inclut l'utilisateur et une paire de tokens.
<code>400 Bad Request</code>	<code>code</code> manquant, Microsoft a rejeté l'échange (<code>code</code> expiré/déjà utilisé, mauvais <code>redirect_uri</code> , consentement révoqué), ou le profil Graph n'a ni <code>mail</code> ni <code>userPrincipalName</code> .

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>code</code>	string	oui	Code d'autorisation obtenu en redirigeant le navigateur vers l'endpoint <code>/oauth2/v2.0/authorize</code> de Microsoft.

Workflow

1. Le frontend redirige le navigateur vers Microsoft pour l'authentification/le consentement.
2. Microsoft redirige vers l'URL de callback du frontend avec `?code=...`.
3. Le frontend envoie `{ code }` en `POST` à cet endpoint.
4. Le backend échange `code` contre un access token Graph via MSAL (côté serveur uniquement - utilise `AZURE_CLIENT_SECRET`, jamais exposé au frontend).
5. Le backend récupère le profil Graph (`givenName`, `surname`, `mail/userPrincipalName`) et vérifie si le compte a une photo.
6. Le backend récupère ou crée le `User` (apparié par email) et sa ligne `OAuthUser(provider="microsoft")` liée.
7. Réponse : `{ "user": {...}, "access": "<jwt>", "refresh": "<jwt>" }`, format identique à `/login/`.

`POST /api/users/oauth/microsoft/link/`

Relie le compte de l'appelant **déjà authentifié** à une identité Microsoft (l'action « connecter mon compte Microsoft » des paramètres) – contrairement à `POST /api/users/oauth/microsoft/` (connexion/inscription, `AllowAnonymous`), cette route cible toujours `request.user` plutôt que d'apparier/créer un compte par email.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Liaison effectuée ; la réponse inclut l'utilisateur mis à jour et un message de confirmation.
400 Bad Request	code manquant, Microsoft a rejeté l'échange (code expiré/déjà utilisé, mauvais <code>redirect_uri</code> , consentement révoqué), le profil Graph n'a pas d'email, ou cet email appartient déjà à un compte différent .
401 Unauthorized	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
code	string	oui	Code d'autorisation obtenu de la même façon que pour la connexion, mais en étant déjà connecté.

Workflow

1. Depuis une page de paramètres, le frontend redirige le navigateur vers Microsoft pour l'authentification/le consentement, comme pour la connexion.
2. Microsoft redirige avec `?code=...` ; le frontend envoie `{ code }` en `POST` ici, avec `Authorization: Bearer <access>` pour la session *courante*.
3. Le backend échange `code` contre un access token Graph et récupère le profil (mêmes fonctions que la route de connexion).
4. Le backend synchronise `request.user.email` avec l'email du profil Microsoft et appelle `set_unusable_password()` sur le compte - **le compte ne peut plus se connecter que via Microsoft à partir de là**.
5. Réponse : `{ "user": {...}, "detail": "Microsoft account linked successfully. You can only sign in with Microsoft from now on." }`.

`POST /api/users/token/refresh/`

Échange un refresh token contre un nouvel access token. Vue Simple JWT intégrée (`TokenRefreshView`), partagée par toutes les méthodes de connexion (mot de passe, Microsoft).

Auth : `AllowAny`

Codes de statut

Statut	Signification
200 OK	<code>{ "access": "<new jwt>" }</code>
401 Unauthorized	refresh invalide, expiré, ou blacklisté (par ex. après <code>/logout/</code>).

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>refresh</code>	string	oui	Un refresh token précédemment émis par inscription/connexion/OAuth.

Workflow

1. Le client détecte (ou anticipe) l'expiration de son token `access`.
2. Le client envoie `{ "refresh": "<refresh>" }` en POST.
3. Le serveur vérifie le token (signature, expiration, non-blacklisté) et émet un nouvel `access` token (`ROTATE_REFRESH_TOKENS` est désactivé, donc le même `refresh` token reste valide et réutilisable pour de futurs rafraîchissements).
4. Le client remplace son `access` token stocké et continue d'utiliser le même `refresh` token.

`POST /api/users/change-password/`

Change le mot de passe de l'utilisateur authentifié lui-même.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	<code>{ "detail": "Password updated successfully." }</code>
400 Bad Request	<code>current_password</code> erroné, <code>new_password</code> rejeté par les validateurs Django, ou compte n'ayant que des identités OAuth liées (pas de mot de passe local à remplacer).
401 Unauthorized	Access token absent/invalid.
405 Method Not Allowed	Toute méthode autre que <code>POST</code> .

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>current_password</code>	string	oui	Doit correspondre au mot de passe actuel du compte.
<code>new_password</code>	string	oui	Validé par les validateurs de mot de passe Django.

Workflow

1. Le client envoie `{ current_password, new_password }` avec un access token valide.
2. Le serializer rejette d'emblée la requête si le compte est OAuth-only (`user.oauth_accounts.exists()`).
3. Le serializer vérifie `current_password` contre le hash stocké.
4. Le serveur appelle `set_password(new_password)` et sauvegarde l'utilisateur.
5. **Remarque :** les access/refresh tokens existants ne sont *pas* révoqués par cet appel - combiner avec `POST /api/users/logout/` si les autres sessions doivent aussi être déconnectées de force.

`POST /api/users/change-email/`

Change l'adresse email de l'utilisateur authentifié lui-même.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	<code>{ "detail": "Email updated successfully.", "email": "<new_email>", "oauth_unlinked": <bool> }</code>
400 Bad Request	<code>new_email</code> déjà utilisé par un autre compte, ou compte OAuth-only avec <code>new_password</code> manquant/rejeté par les validateurs Django.
401 Unauthorized	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>new_email</code>	string	oui	Doit être unique.
<code>new_password</code>	string	requis uniquement si le compte est OAuth-only	Devient le nouveau mot de passe local du compte.

Workflow

1. Le client envoie `{ new_email }` (plus `new_password` si le compte n'a pas de mot de passe local).
2. Si le compte n'a que des identités OAuth liées (`user.oauth_accounts.exists()`), `new_password` est requis : la connexion OAuth Microsoft apparie les utilisateurs par email, donc le changer ici désynchroniserait sinon silencieusement les deux. Le mot de passe est défini et **chaque ligne OAuthUser liée est supprimée** (`oauth_unlinked: true` dans la réponse) - le compte devient local uniquement.
3. Pour un compte déjà local, seul l'email change (`oauth_unlinked: false`).

POST /api/users/change-avatar/

Change l'avatar/l'icône de profil de l'utilisateur authentifié lui-même - y compris pour les comptes OAuth, où elle remplace l'avatar synchronisé à l'origine depuis le fournisseur.

Auth : IsAuthenticated

Codes de statut

Statut	Signification
200 OK	{ "detail": "Avatar updated successfully.", "profile_icon": "<data URI>" }
400 Bad Request	Type d'image non autorisé (PNG/JPEG/SVG uniquement), ou data URI malformée.
401 Unauthorized	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
avatar	String	oui	Data URI en base64 (PNG, JPEG ou SVG uniquement - <code>common.image_validation.validate_image_data_uri</code>).

Workflow

1. Le client envoie { "avatar": "data:image/png;base64,..." } en POST.
2. Le serveur valide le type MIME/l'encodage, stocke la data URI directement sur `profile_icon`, et la renvoie.

`GET /api/users/me/`

Renvoie le profil de l'utilisateur authentifié lui-même.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	La représentation <code>User</code> de l'appelant.
401 Unauthorized	Access token absent/invalid/expiré/blacklisté.

Workflow

1. Le client envoie `GET` avec `Authorization: Bearer <access>`.
2. Le serveur renvoie directement `UserSerializer(request.user).data` - aucune requête supplémentaire n'est nécessaire puisque l'utilisateur est déjà résolu par l'authentification.

`GET /api/users/`

Liste tous les comptes utilisateurs. Réservé au staff - c'est un listing administratif, pas un annuaire public.

Auth : `IsAuthenticated + IsAdminUser`

Codes de statut

Statut	Signification
200 OK	Tableau JSON simple d'utilisateurs (aucune pagination configurée sur ce viewset).
401 Unauthorized	Access token absent/invalid.
403 Forbidden	Authentifié mais pas staff.

Workflow

1. Un client staff envoie `GET` sur `/api/users/`.
2. `ViewSet.get_permissions()` exige `IsAdminUser` pour `list/retrieve`.
3. Le serveur renvoie chaque ligne `User`, sérialisée avec `UserSerializer`.

`GET /api/users/{id}/`

Récupère le profil d'un utilisateur par son id. Réservé au staff, même logique que la route de liste.

Auth : `IsAuthenticated` + `IsAdminUser`

Codes de statut

Statut	Signification
200 OK	L'utilisateur demandé.
401 Unauthorized	Access token absent/invalid.
403 Forbidden	Authentifié mais pas staff.
404 Not Found	Aucun utilisateur avec cet id.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id de l'utilisateur ciblé.

Workflow : identique à la liste, restreint à une seule ligne via `get_object()`.

`PATCH / PUT /api/users/{id}/`

Met à jour les champs du profil d'un utilisateur (le sien, ou celui de n'importe qui si staff).

Auth : `IsAuthenticated` + `IsSelfOrStaff`

Codes de statut

Statut	Signification
200 OK	Représentation <code>User</code> mise à jour.
400 Bad Request	Valeur de champ invalide (par ex. email/username en doublon).
401 Unauthorized	Access token absent/invalide.
403 Forbidden	L'appelant n'est ni l'utilisateur ciblé, ni staff.
404 Not Found	Aucun utilisateur avec cet id.

Paramètres (chemin + corps)

Nom	Emplacement	Type	Requis	Description
<code>id</code>	chemin	integer	oui	Id de l'utilisateur ciblé.
<code>username, email, first_name, last_name, profile_icon</code>	corps	variable	non (tout sous-ensemble pour <code>PATCH</code> ; tous les champs modifiables pour <code>PUT</code>)	Champs à modifier.

Workflow

1. Le client envoie `PATCH` (partiel) ou `PUT` (complet) avec les champs à modifier.
2. `IsSelfOrStaff.has_object_permission` vérifie `request.user.is_staff or obj == request.user`.
3. Le serveur valide et sauvegarde via `UserSerializer`.

`DELETE /api/users/{id}/`

Supprime son propre compte (ou n'importe quel compte, si staff).

Auth : `IsAuthenticated` + `IsSelfOrStaff`

Codes de statut

Statut	Signification
204 No Content	Compte supprimé.
401 Unauthorized	Access token absent/invalid.
403 Forbidden	L'appelant n'est ni l'utilisateur ciblé, ni staff.
404 Not Found	Aucun utilisateur avec cet id.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id de l'utilisateur ciblé.

Workflow

1. Le client envoie `DELETE` sur `/api/users/{id}/`.
2. Vérification de permission comme ci-dessus.
3. `ViewSet.perform_destroy()` appelle `users.services.delete_user_account()`, qui - dans une seule transaction - dénoue chaque clé étrangère `PROTECT` qui bloquerait sinon la suppression, dans l'ordre des dépendances : messages (rédigés par l'utilisateur, ou postés dans un cookbook/une recette qu'il possède), liens de repas planifiés, ingrédients/tags/étapes/favoris des recettes, préférences de cuisine, partages de cookbooks, puis les recettes/plannings/cookbooks de l'utilisateur lui-même, ses lignes `OAuthUser`, et enfin la ligne `User`.
4. C'est une véritable suppression en cascade, pas une réaffectation : les recettes/plannings/messages créés par l'utilisateur - y compris ceux classés dans un cookbook qu'il possède, même rédigés par quelqu'un d'autre - sont définitivement supprimés avec le compte. Il n'y a pas d'option « conserver en tant qu'orphelin/personnel » ici, contrairement à la suppression d'un cookbook (voir [DELETE /api/cookbooks/{id}/](#), où les recettes/plannings survivent).

2. Cookbooks (cookbooks)

Les cookbooks constituent la frontière de partage/permission pour les recettes, plannings et messages. Le créateur d'un cookbook en est l'administrateur implicite ; d'autres utilisateurs peuvent se voir accorder un rôle `creator` / `editor` / `commentator` / `reader` via [share](#). Voir `cookbooks.permissions` pour l'ordre exact des rangs.

`GET /api/cookbooks/`

Liste les cookbooks que l'appelant a créés **ou** avec lesquels il a été partagé.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Liste paginée (voir Pagination).
401 Unauthorized	Access token absent/invalid.

Paramètres (query)

Nom	Type	Description
<code>name</code>	<code>string</code>	Correspondance partielle insensible à la casse sur le nom du cookbook.
<code>shared_with_me</code>	<code>boolean</code>	<code>true</code> : uniquement les cookbooks partagés avec l'appelant (dont il n'est pas propriétaire). <code>false</code> : uniquement ses propres cookbooks.
<code>role</code>	<code>string</code>	Une valeur parmi <code>reader</code> , <code>commentator</code> , <code>editor</code> , <code>creator</code> . Uniquement les cookbooks partagés avec l'appelant à exactement ce rôle - <code>admin</code> n'est pas une valeur valide ici puisqu'il n'est jamais stocké (implicite pour le créateur du cookbook) : filtrer par un rôle réel exclut donc déjà ses propres cookbooks.
<code>page</code>	<code>integer</code>	Numéro de page.
<code>page_size</code>	<code>integer</code>	Éléments par page (100 maximum).

Workflow

1. `get_queryset()` restreint les lignes à `creator=user` OR `shared_with__user=user`.
2. Les filtres et la pagination s'appliquent ensuite sur cet ensemble de base.
3. Chaque cookbook est renvoyé avec ses `recipes`, `plannings` et sa liste `shared_with` imbriqués.

`POST /api/cookbooks/`

Crée un cookbook. L'appelant en devient le créateur (administrateur implicite).

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
<code>201 Created</code>	Cookbook créé.
<code>400 Bad Request</code>	name manquant/invalid.
<code>401 Unauthorized</code>	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>name</code>	String	oui	
<code>icon</code>	String	non	Data URI/image en base64 ; une icône par défaut est utilisée si omis.

Workflow

- `CookbookWriteSerializer` valide `name/icon`.
- `perform_create()` définit `creator=request.user`.
- La réponse utilise le format imbriqué complet `CookbookSerializer` (`recipes/plannings/shared_with` vides pour un cookbook tout neuf).

GET /api/cookbooks/{id}/

Récupère un cookbook avec ses recettes, plannings et membres.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Le cookbook.
401 Unauthorized	Access token absent/invalid.
404 Not Found	Le cookbook n'existe pas, ou l'appelant n'en est pas le créateur et n'est pas dans <code>shared_with</code> (un non-membre reçoit un 404, jamais un 403, afin de ne pas pouvoir confirmer que le cookbook existe).

Paramètres (chemin)

Nom	Type	Description
id	integer	Id du cookbook.

PATCH / PUT /api/cookbooks/{id}/

Renomme un cookbook / change son icône. Réservé à l'admin.

Auth : `IsAuthenticated` + `IsCookbookAdmin`

Codes de statut

Statut	Signification
200 OK	Cookbook mis à jour.
400 Bad Request	Valeur de champ invalide.
401 Unauthorized	Access token absent/invalid.
403 Forbidden	L'appelant a accès au cookbook (sinon 404) mais n'en est pas le créateur/staff.
404 Not Found	Pas du tout membre.

Paramètres (chemin + corps)

Nom	Emplacement	Requis	Description
<code>id</code>	chemin	oui	Id du cookbook.
<code>name, icon</code>	corps	non (tout sous-ensemble pour <code>PATCH</code>)	Champs à modifier.

`DELETE /api/cookbooks/{id}/`

Supprime un cookbook. Réservé à l'admin.

Auth : `IsAuthenticated` + `IsCookbookAdmin`

Codes de statut

Statut	Signification
<code>204 No Content</code>	Supprimé.
<code>401 Unauthorized</code>	Access token absent/invalid.
<code>403 Forbidden</code>	Membre mais pas admin.
<code>404 Not Found</code>	Pas membre.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id du cookbook.

Workflow

1. `CookbookViewSet.perform_destroy()` dénoue lui-même les FK `PROTECT` du cookbook, dans une seule transaction, si bien que cette route ne renvoie plus jamais de `500` en usage normal :

- `Recipe.cookbook` / `Planning.cookbook` sont **vidés, pas mis en cascade** - chaque recette/planning classé dans le cookbook survit en tant qu'élément personnel autonome, possédé par celui qui l'a créé.
- Les lignes `SharedUserCookbook` (partages) et `Message` (les deux canaux) du cookbook sont supprimées purement et simplement, car elles n'ont de sens que dans le contexte de ce cookbook.

2. Cette route n'a pas d'option « supprimer aussi les recettes/plannings ». Pour en supprimer certains en même temps que le cookbook plutôt que de les conserver, il faut supprimer ces recettes/plannings

spécifiques au préalable (via leurs propres routes `DELETE`) *avant* d'appeler celle-ci - c'est ce que fait la confirmation de suppression de cookbook du frontend.

`POST / PATCH /api/cookbooks/{id}/share/`

Accorde ou met à jour l'accès d'un ou plusieurs utilisateurs au cookbook, en un seul appel. Les deux méthodes se comportent de façon identique (un upsert) ; `PATCH` se lit simplement mieux quand on modifie le rôle d'un membre existant.

Auth : `IsAuthenticated` + `IsCookbookAdmin`

Codes de statut

Statut	Signification
200 OK	Le cookbook est renvoyé avec sa liste <code>shared_with</code> à jour.
400 Bad Request	Ni/à la fois <code>user/email</code> fournis par entrée, email inconnu, <code>role</code> invalide, ou tentative de partage avec le créateur du cookbook lui-même.
401 Unauthorized	Access token absent/invalide.
403 Forbidden	Membre mais pas admin.
404 Not Found	Pas membre du cookbook.

Paramètres (chemin + corps)

Nom	Emplacement	Type	Requis	Description
<code>id</code>	chemin	integer	oui	Id du cookbook.
<code>shares</code>	corps	array	oui	Liste d'entrées { <code>user</code> ou <code>email</code> , <code>role</code> }.
<code>shares[].user</code>	corps	integer	un des deux (<code>user/email</code>)	Id de l'utilisateur cible.
<code>shares[].email</code>	corps	string	un des deux (<code>user/email</code>)	Email de l'utilisateur cible (résolu en utilisateur côté serveur).
<code>shares[].role</code>	corps	string	oui	Une valeur parmi <code>creator</code> , <code>editor</code> , <code>commentator</code> , <code>reader</code> (du plus au

Nom	Emplacement	Type	Requis	Description
				moins permissif ; il n'y a pas de rôle admin ici).

Workflow

1. L'admin envoie POST/PATCH avec `{ "shares": [{ "user": 2, "role": "editor" }, ...] }`.
2. Chaque entrée est validée : exactement un des deux champs `user/email`, `email` résolu en `User`, la cible n'est pas le créateur du cookbook.
3. Pour chaque entrée, `SharedUserCookbook.objects.update_or_create(cookbook, user, defaults={"role": ...})` - repartager avec un membre existant **met à jour son rôle** plutôt que de dupliquer la ligne.
4. La réponse renvoie le cookbook avec sa liste `shared_with` rafraîchie.

`POST /api/cookbooks/{id}/unshare/`

Révoque l'accès d'un ou plusieurs utilisateurs au cookbook, en un seul appel.

Auth : `IsAuthenticated` + `IsCookbookAdmin`

Codes de statut

Statut	Signification
200 OK	Le cookbook est renvoyé avec sa liste <code>shared_with</code> à jour (révoquer un non-membre est un no-op, pas une erreur).
400 Bad Request	Id(s) utilisateur invalide(s) dans <code>users</code> .
401 Unauthorized	Access token absent/invalide.
403 Forbidden	Membre mais pas admin.
404 Not Found	Pas membre du cookbook.

Paramètres (chemin + corps)

Nom	Emplacement	Type	Requis	Description
<code>id</code>	chemin	integer	oui	Id du cookbook.
<code>users</code>	corps	array d'integers	oui	Ids des utilisateurs à révoquer.

`GET /api/cookbooks/{id}/export/`

Exporte un cookbook (ses recettes et plannings) en JSON portable.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
<code>200 OK</code>	Export JSON, avec un en-tête <code>Content-Disposition: attachment</code> .
<code>401 Unauthorized</code>	Access token absent/invalid.
<code>404 Not Found</code>	Pas membre du cookbook.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id du cookbook.

Workflow

1. N'importe quel membre (n'importe quel rôle, y compris `reader`) peut exporter.
2. Chaque recette rangée dans le cookbook est incluse (même format qu'un export de recette), chacune associée à un `id` local à l'export.
3. Chaque planning référence ses recettes programmées via `recipe_id`, correspondant à un de ces `id` locaux - un repas programmant une recette qui n'est *pas* rangée dans ce cookbook est silencieusement omis.
4. `shared_with` (les membres) n'est jamais inclus dans l'export.
5. Le résultat peut être renvoyé tel quel à `POST /api/cookbooks/import/`.

GET /api/cookbooks/export/

Exporte tous les cookbooks **créés par l'appelant** sous forme de tableau JSON.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Tableau JSON, un objet d'export par cookbook possédé (les cookbooks uniquement partagés sont exclus).
401 Unauthorized	Access token absent/invalid.

POST /api/cookbooks/import/

Importe un ou plusieurs cookbooks à partir de JSON préalablement exporté.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
201 Created	Renvoie toujours un tableau JSON des cookbooks créés, même pour une charge utile contenant un seul objet.
400 Bad Request	Un élément quelconque échoue à la validation (par ex. un <code>meals[].recipe_id</code> ne correspondant à aucun <code>recipes[].id</code>) - tout l'import est rejeté, rien n'est créé.
401 Unauthorized	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<i>(corps de requête)</i>	objet ou tableau	oui	Un objet d'export de cookbook, ou un tableau JSON de ceux-ci (tel que produit par les deux routes d'export ci-dessus).

Workflow

1. Le client envoie exactement le JSON précédemment téléchargé depuis une route d'export.
2. Pour chaque objet cookbook : un nouveau `Cookbook` est créé avec l'appelant comme créateur ; chaque recette imbriquée est créée (ingrédients/tags appariés par nom et réutilisés s'ils existent déjà, sinon créés ; les étapes sont toujours créées à neuf) ; les repas de chaque planning sont reconstruits en résolvant `recipe_id` par rapport aux valeurs `recipes[].id` propres à la charge utile.
3. Aucun membre n'est importé - seul l'utilisateur important a accès au(x) nouveau(x) cookbook(s).
4. L'opération entière est atomique : tout échec de validation annule tout.

3. Recettes, tags & ingrédients (recipes)

GET /api/recipes/

Liste les recettes visibles par l'appelant : recettes personnelles (sans= cookbook), recettes qu'il a créées, ou recettes rangées dans un cookbook qu'il possède ou avec lequel il a été partagé.

Auth : IsAuthenticated

Codes de statut

Statut	Signification
200 OK	Liste paginée.
401 Unauthorized	Access token absent/invalid.

Paramètres (query)

Nom	Type	Description
name	string	Correspondance partielle insensible à la casse (ILIKE SQL) sur le titre de la recette.
tags	string	Noms et/ou ids de tags séparés par des virgules ; la recette doit posséder tous les tags listés.
ingredients	string	Noms et/ou ids d'ingrédients séparés par des virgules ; la recette doit contenir tous les ingrédients listés.
cookbook	string	Correspondance partielle insensible à la casse sur le nom du cookbook.
in_cookbook	boolean	true : uniquement les recettes rangées dans un cookbook. false : uniquement les recettes autonomes.
planning	string	Correspondance partielle insensible à la casse sur le nom d'un planning dans lequel la recette est programmée.
in_planning	boolean	true : uniquement les recettes programmées dans au moins un planning. false : uniquement celles qui ne le sont dans aucun.

Nom	Type	Description
<code>favorite</code>	boolean	<code>true</code> : uniquement les recettes favorites de l'appelant. <code>false</code> : uniquement celles qui ne le sont pas.
<code>shared_with_me</code>	boolean	<code>true</code> : uniquement les recettes rangées dans un cookbook partagé avec l'appelant (dont il n'est pas propriétaire).
<code>prep_time_min</code> / <code>prep_time_max</code>	number	Bornes (en minutes) sur la somme des durées des étapes de la recette.
<code>cooking_duration_min</code> / <code>cooking_duration_max</code>	number	Bornes (en minutes) sur le champ <code>cooking_duration</code> .
<code>page</code> / <code>page_size</code>	integer	Pagination.

POST /api/recipes/

Crée une recette avec ses ingrédients, tags et étapes.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
<code>201 Created</code>	Recette créée.
<code>400 Bad Request</code>	Champ manquant/invalid, ou <code>cookbook</code> fourni mais l'appelant n'a pas au moins le rang <code>creator</code> dessus.
<code>401 Unauthorized</code>	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>title</code>	string	oui	

Nom	Type	Requis	Description
<code>image, source</code>	string	non	
<code>cooking_duration</code>	decimal	non	En minutes.
<code>cookbook</code>	integer	non	Id du cookbook cible ; l'appelant doit avoir le rôle <code>creator</code> ou en être l'admin.
<code>ingredients</code>	array	non	{ <code>name</code> , <code>image?</code> , <code>quantity</code> , <code>unity?</code> , <code>person_numbers</code> } par ligne ; apparié/réutilisé par nom (insensible à la casse), créé si nouveau.
<code>tags</code>	array	non	{ <code>name</code> , <code>type</code> , <code>description?</code> } par tag ; apparié/réutilisé par nom, créé si nouveau.
<code>steps</code>	array	non	{ <code>description</code> , <code>step_number</code> , <code>dury</code> , <code>type</code> } par étape ; toujours créées à neuf.

Workflow

1. `RecipeWriteSerializer` valide les champs de la recette et, si `cookbook` est renseigné, que l'appelant a au moins le rang `creator` dessus (`cookbooks.permissions.has_rank`).
2. `creator` est défini à partir de la requête, jamais depuis la charge utile.
3. Ingrédients/tags sont synchronisés par nom (get-or-create dans le catalogue partagé) ; les étapes sont (re)créées à neuf (`recipes/services.py`).
4. La réponse utilise le format imbriqué en lecture seule `RecipeSerializer`.

`GET /api/recipes/{id}/`

Récupère une recette avec ses ingrédients, tags et étapes.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
<code>200 OK</code>	La recette.
<code>401 Unauthorized</code>	Access token absent/invalid.

Statut	Signification
403 Forbidden	Recette rangée dans un cookbook où l'appelant n'a aucun rôle (visible dans le queryset de base seulement si créateur/membre du cookbook).
404 Not Found	La recette n'existe pas / n'est pas visible selon <code>get_queryset()</code> .

Paramètres (chemin)

Nom	Type	Description
id	integer	Id de la recette.

PATCH / PUT /api/recipes/{id}/

Met à jour les champs d'une recette et/ou remplace ses ingrédients/tags/étapes.

Auth : `IsAuthenticated` + `CookbookItemPermission`

Codes de statut

Statut	Signification
200 OK	Recette mise à jour.
400 Bad Request	Champ invalide, ou rang insuffisant sur le <code>cookbook</code> cible.
401 Unauthorized	Access token absent/invalid.
403 Forbidden	L'appelant n'est pas le créateur de la recette et n'a pas au moins le rang <code>editor</code> sur son cookbook (ou rang <code>creator</code> + requis pour une recette autonome sans cookbook - seul le créateur peut la toucher).
404 Not Found	La recette n'existe pas / n'est pas visible.

Paramètres (chemin + corps)

Nom	Emplacement	Requis	Description
id	chemin	oui	Id de la recette.

Nom	Emplacement	Requis	Description
title, image, source, cooking_duration, cookbook, ingredients, tags, steps	corps	non (tout sous-ensemble)	Omettre ingredients/tags/steps les laisse inchangés ; passer [] les vide.

DELETE /api/recipes/{id}/

Supprime une recette, en déliant d'abord ses ingrédients/tags/étapes/favoris (toutes des FK PROTECT vers Recipe).

Auth : IsAuthenticated + CookbookItemPermission

Codes de statut

Statut	Signification
204 No Content	Supprimée.
401 Unauthorized	Access token absent/invalid.
403 Forbidden	L'appelant n'est pas le créateur et n'a pas le rang creator sur son cookbook.
404 Not Found	La recette n'existe pas / n'est pas visible.

Paramètres (chemin)

Nom	Type	Description
id	integer	Id de la recette.

GET /api/recipes/{id}/export/

Exporte une recette en JSON portable (sans id/creator/cookbook).

Auth : IsAuthenticated

Codes de statut

Statut	Signification
200 OK	Export JSON, avec un en-tête Content-Disposition: attachment.
401 Unauthorized	Access token absent/invalid.

Statut	Signification
404 Not Found	La recette n'existe pas / n'est pas visible.

Paramètres (chemin)

Nom	Type	Description
id	integer	Id de la recette.

Workflow : toute recette que l'appelant peut lire (la sienne, ou dans un cookbook partagé) peut être exportée ainsi ; le résultat peut être renvoyé tel quel à [POST /api/recipes/import/](#), et atterrira comme recette personnelle autonome.

GET /api/recipes/export/

Exporte les recettes **personnelles** de l'appelant (créées par lui, sans cookbook) sous forme de tableau JSON.

Auth : IsAuthenticated

Codes de statut

Statut	Signification
200 OK	Tableau JSON.
401 Unauthorized	Access token absent/invalid.

POST /api/recipes/import/

Importe une ou plusieurs recettes à partir de JSON préalablement exporté.

Auth : IsAuthenticated

Codes de statut

Statut	Signification
201 Created	Renvoie toujours un tableau JSON des recettes créées, même pour une charge utile contenant un seul objet.
400 Bad Request	Un élément quelconque échoue à la validation - tout l'import est rejeté.
401 Unauthorized	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
(corps de requête)	objet ou tableau	oui	Un objet d'export de recette, ou un tableau JSON de ceux-ci.

Workflow : chaque recette importée devient toujours une recette personnelle autonome possédée par l'appelant (`cookbook` n'est jamais défini, même si l'export en provenait à l'origine) ; ingrédients/tags appariés/réutilisés par nom (insensible à la casse), créés si nouveaux ; étapes toujours créées à neuf.

`POST / DELETE /api/recipes/{id}/favorite/`

Ajoute (`POST`) ou retire (`DELETE`) une recette des favoris de l'appelant.

Auth : `IsAuthenticated + CookbookItemPermission` (l'accès en lecture suffit - ajouter en favori ne requiert pas de droits d'édition)

Codes de statut

Statut	Signification
201 Created	<code>POST</code> , et la recette n'était pas déjà en favori.
200 OK	<code>POST</code> , et la recette était déjà en favori (idempotent, aucun doublon créé).
204 No Content	<code>DELETE</code> (idempotent, quel que soit l'état favori précédent).
401 Unauthorized	Access token absent/invalid.
404 Not Found	La recette n'existe pas / n'est pas visible par l'appelant.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id de la recette.

`GET /api/tags/` et `GET /api/tags/{id}/`

Consultation en lecture seule du catalogue partagé de tags (catégories/sous-catégories). Les tags ne sont jamais créés que via les endpoints d'écriture des recettes.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Liste (tableau JSON simple - non paginé) ou tag unique.
401 Unauthorized	Access token absent/invalid.
404 Not Found	(route de détail) Aucun tag avec cet id.

Paramètres (query, route de liste uniquement)

Nom	Type	Description
search	string	Correspondance de sous-chaîne sur <code>name</code> (<code>SearchFilter</code> de DRF).
type	string	Correspondance exacte sur la sous-catégorie du tag (par ex. "repas", "regime_alimentaire").

GET /api/ingredients/ et GET /api/ingredients/{id}/

Consultation en lecture seule du catalogue partagé d'ingrédients (par ex. pour la recherche/"autocomplétion"). Les ingrédients ne sont jamais créés que via les endpoints d'écriture des recettes.

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
200 OK	Liste (tableau JSON simple - non paginé) ou ingrédient unique.
401 Unauthorized	Access token absent/invalid.
404 Not Found	(route de détail) Aucun ingrédient avec cet id.

Paramètres (query, route de liste uniquement)

Nom	Type	Description
search	string	Correspondance de sous-chaîne sur <code>name</code> (<code>SearchFilter</code> de DRF).

4. Planning (planning)

Les plannings programment des recettes dans des créneaux jour/moment-du-repas/plat (dayofweek × lunch × type, jusqu'à 42 créneaux par semaine). Même modèle de visibilité/permission que les recettes : un planning hors de tout cookbook n'est gérable que par son créateur ; un planning rangé dans un cookbook est soumis au rôle de l'appelant sur ce cookbook.

GET /api/plannings/

Auth : IsAuthenticated

Codes de statut

Statut	Signification
200 OK	Liste paginée.
401 Unauthorized	Access token absent/invalid.

Paramètres (query)

Nom	Type	Description
name	string	Correspondance partielle insensible à la casse sur le nom du planning.
type	string	Correspondance exacte sur le type du planning : journalier ou hebdomadaire.
cookbook	string	Correspondance partielle insensible à la casse sur le nom du cookbook.
in_cookbook	boolean	true : uniquement les plannings rangés dans un cookbook. false : uniquement les plannings autonomes.
shared_with_me	boolean	true : uniquement les plannings dans un cookbook partagé avec l'appelant.
page / page_size	integer	Pagination.

`POST /api/plannings/`

Auth : `IsAuthenticated`

Codes de statut

Statut	Signification
<code>201 Created</code>	Planning créé.
<code>400 Bad Request</code>	Champ manquant/invalid, rang insuffisant sur <code>cookbook</code> , ou deux repas ciblant le même créneau (<code>dayofweek</code> , <code>lunch</code> , <code>type</code>).
<code>401 Unauthorized</code>	Access token absent/invalid.

Paramètres (corps, JSON)

Nom	Type	Requis	Description
<code>name</code>	string	oui	
<code>icon</code>	string	non	Une icône par défaut est utilisée si omise.
<code>cookbook</code>	integer	non	L'appelant doit avoir le rôle <code>creator</code> ou en être l'admin.
<code>meals</code>	array	non	{ <code>recipe</code> , <code>dayofweek</code> , <code>lunch</code> , <code>type</code> } par repas programmé. Au plus une recette par créneau (<code>dayofweek</code> , <code>lunch</code> , <code>type</code>).

Paramètres (valeurs possibles de `meals[]`)

Champ	Valeurs autorisées
<code>lunch</code>	<code>midi</code> , <code>soir</code>
<code>type</code>	<code>entree</code> , <code>plat</code> , <code>dessert</code>
<code>dayofweek</code>	<code>lundi</code> , <code>mardi</code> , <code>mercredi</code> , <code>jeudi</code> , <code>vendredi</code> , <code>samedi</code> , <code>dimanche</code>

GET /api/plannings/{id}/

Auth : IsAuthenticated

Codes de statut

Statut	Signification
200 OK	Le planning, avec ses <code>meals</code> programmés.
401 Unauthorized	Access token absent/invalid.
404 Not Found	N'existe pas / non visible.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id du planning.

PATCH / PUT /api/plannings/{id}/

Auth : IsAuthenticated + CookbookItemPermission

Codes de statut

Statut	Signification
200 OK	Planning mis à jour.
400 Bad Request	Champ invalide, conflit de créneau, ou rang insuffisant sur <code>cookbook</code> .
401 Unauthorized	Access token absent/invalid.
403 Forbidden	Rang insuffisant (au moins <code>editor</code> requis ; <code>creator</code> pour un planning autonome).
404 Not Found	N'existe pas / non visible.

Paramètres (chemin + corps)

Nom	Emplacement	Requis	Description
<code>id</code>	chemin	oui	Id du planning.
<code>name, icon, cookbook, meals</code>	corps	non (tout sous-ensemble)	Omettre <code>meals</code> laisse le planning existant inchangé ; passer <code>[]</code> le vide.

DELETE `/api/plannings/{id}/`

Délie les repas programmés (`RecipePlanning`, `FK PROTECT`) avant de supprimer le planning lui-même.

Auth : `IsAuthenticated` + `CookbookItemPermission`

Codes de statut

Statut	Signification
<code>204 No Content</code>	Supprimé.
<code>401 Unauthorized</code>	Access token absent/invalid.
<code>403 Forbidden</code>	Rang insuffisant (<code>creator</code> requis).
<code>404 Not Found</code>	N'existe pas / non visible.

Paramètres (chemin)

Nom	Type	Description
<code>id</code>	integer	Id du planning.

5. Messagerie (messaging)

Les messages existent en trois variantes de la même ressource sous-jacente : le **canal global** d'un cookbook, le canal d'une **recette précise**, et le canal d'un **planning précis** - ces deux derniers toujours rattachés à un cookbook. Il n'existe volontairement aucune route de modification – un message ne peut être que posté ou supprimé, jamais édité.

GET /api/cookbooks/{cookbook_pk}/messages/

Liste les messages du canal global du cookbook (*recipe* toujours *null*).

Auth : `IsAuthenticated` + `CanAccessCookbookMessages` (tout rôle, y compris `reader`, peut lire)

Codes de statut

Statut	Signification
200 OK	Liste paginée.
401 Unauthorized	Access token absent/invalid.
404 Not Found	<code>cookbook_pk</code> n'existe pas, ou l'appelant n'est pas membre (et n'est pas staff) - un tiers ne peut pas savoir que le cookbook existe.

Paramètres (chemin + query)

Nom	Emplacement	Type	Description
<code>cookbook_pk</code>	chemin	integer	Id du cookbook.
<code>page / page_size</code>	query	integer	Pagination.

POST /api/cookbooks/{cookbook_pk}/messages/

Poste un message dans le canal global du cookbook.

Auth : `IsAuthenticated` + `CanAccessCookbookMessages` (au moins le rôle `commentator` requis pour écrire)

Codes de statut

Statut	Signification
201 Created	Message créé.
400 Bad Request	<code>content/canal</code> manquant/invalid.
401 Unauthorized	Access token absent/invalid.

Statut	Signification
403 Forbidden	Membre avec seulement le rôle <code>reader</code> (accès lecture, pas d'écriture).
404 Not Found	<code>cookbook_pk</code> n'existe pas / l'appelant n'est pas membre.

Paramètres (chemin + corps)

Nom	Emplacement	Type	Requis	Description
<code>cookbook_pk</code>	chemin	integer	oui	Id du cookbook.
<code>content</code>	corps	string	oui	Texte du message.
<code>canal</code>	corps	string	oui	Libellé libre du canal de conversation.

Workflow : `author`, `cookbook` (depuis l'URL) et `recipe=None` sont toujours définis côté serveur, jamais acceptés depuis le corps de la requête.

`GET /api/cookbooks/{cookbook_pk}/messages/{pk}/`

Récupère un message du canal global du cookbook.

Auth : `IsAuthenticated` + `CanAccessCookbookMessages`

Codes de statut

Statut	Signification
200 OK	Le message.
401 Unauthorized	Access token absent/invalid.
404 Not Found	<code>cookbook_pk/pk</code> n'existe pas, ou non visible.

Paramètres (chemin)

Nom	Type	Description
<code>cookbook_pk</code>	integer	Id du cookbook.

Nom	Type	Description
<code>pk</code>	integer	Id du message.

`DELETE /api/cookbooks/{cookbook_pk}/messages/{pk}/`

Supprime un message du canal global du cookbook.

Auth : `IsAuthenticated` + `CanAccessCookbookMessages` + `CanDeleteMessage`

Codes de statut

Statut	Signification
<code>204 No Content</code>	Supprimé.
<code>401 Unauthorized</code>	Access token absent/invalid.
<code>403 Forbidden</code>	L'appelant n'est ni l'auteur du message, ni l'admin du cookbook, ni staff.
<code>404 Not Found</code>	<code>cookbook_pk/pk</code> n'existe pas, ou non visible.

Paramètres (chemin)

Nom	Type	Description
<code>cookbook_pk</code>	integer	Id du cookbook.
<code>Pk</code>	integer	Id du message.

`GET / POST /api/cookbooks/{cookbook_pk}/recipes/{recipe_pk}/messages/`

Même comportement que les routes de liste/création au niveau cookbook ci-dessus, mais restreint au canal d'une recette précise plutôt qu'au canal global du cookbook.

Auth : identique aux équivalents au niveau cookbook.

Codes de statut : identiques aux deux routes ci-dessus, plus :

Statut	Signification
404 Not Found	Également renvoyé si <code>recipe_pk</code> n'appartient pas à <code>cookbook_pk</code> (<code>recipe.cookbook_id != cookbook_pk</code>) - une paire incohérente ne peut pas servir à divulguer ou mal ranger un message.

Paramètres (chemin + query/corps)

Nom	Emplacement	Type	Description
<code>cookbook_pk</code>	chemin	integer	Id du cookbook.
<code>recipe_pk</code>	chemin	integer	Id de la recette ; doit être rangée dans <code>cookbook_pk</code> .
<code>content, canal</code>	corps (POST uniquement)	string	Identique à la route au niveau cookbook.
<code>page</code> / <code>page_size</code>	query (GET uniquement)	integer	Pagination.

`GET /api/cookbooks/{cookbook_pk}/recipes/{recipe_pk}/messages/{pk}/`

Récupère un message du canal de la recette.

Auth/codes de statut : identiques à la route de récupération au niveau cookbook ci-dessus, plus le cas 404 de non-correspondance `recipe_pk`.

Paramètres (chemin)

Nom	Type	Description
<code>cookbook_pk</code>	integer	Id du cookbook.
<code>recipe_pk</code>	integer	Id de la recette.
<code>pk</code>	integer	Id du message.

`DELETE /api/cookbooks/{cookbook_pk}/recipes/{recipe_pk}/messages/{pk}/`

Supprime un message du canal de la recette.

Auth/codes de statut : identiques à la route de suppression au niveau cookbook ci-dessus, plus le cas 404 de non-correspondance `recipe_pk`.

Paramètres (chemin)

Nom	Type	Description
<code>cookbook_pk</code>	Integer	Id du cookbook.
<code>recipe_pk</code>	integer	Id de la recette.
<code>Pk</code>	integer	Id du message.

GET / POST /api/cookbooks/{cookbook_pk}/plannings/{planning_pk}/messages/

Même comportement que les routes de liste/création au niveau cookbook ci-dessus, mais rattaché au canal d'un planning précis plutôt qu'au canal global du cookbook.

Auth : identique aux équivalents au niveau cookbook.

Codes de statut : identiques aux deux routes au niveau cookbook ci-dessus, plus :

Statut	Signification
404 Not Found	Également renvoyé si <code>planning_pk</code> n'appartient pas à <code>cookbook_pk</code> (<code>planning.cookbook_id != cookbook_pk</code>) - une paire non correspondante ne peut pas servir à divulguer ou mal classer un message.

Paramètres (chemin + query/corps)

Nom	Emplacement	Type	Description
<code>cookbook_pk</code>	chemin	integer	Id du cookbook.
<code>planning_pk</code>	chemin	integer	Id du planning ; doit être rangé dans <code>cookbook_pk</code> .
<code>content, canal</code>	corps (POST uniquement)	string	Identique à la route au niveau cookbook.
<code>page</code> <code>page_size</code>	/ query (GET uniquement)	integer	Pagination.

GET /api/cookbooks/{cookbook_pk}/plannings/{planning_pk}/messages/{pk}/

Récupère un message du canal du planning.

Auth/codes de statut : identiques à la route de récupération au niveau cookbook ci-dessus, plus le cas 404 de non-correspondance `planning_pk`.

Paramètres (chemin)

Nom	Type	Description
<code>cookbook_pk</code>	integer	Id du cookbook.
<code>planning_pk</code>	integer	Id du planning.
<code>pk</code>	integer	Id du message.

DELETE /api/cookbooks/{cookbook_pk}/plannings/{planning_pk}/messages/{pk}/

Supprime un message du canal du planning.

Auth/codes de statut : identiques à la route de suppression au niveau cookbook ci-dessus, plus le cas 404 de non-correspondance `planning_pk`.

Paramètres (chemin)

Nom	Type	Description
<code>cookbook_pk</code>	integer	Id du cookbook.
<code>planning_pk</code>	integer	Id du planning.
<code>pk</code>	integer	Id du message.

Oauth

Cette partie explique spécifiquement comment la connexion OAuth (actuellement : Microsoft / Entra ID) est intégrée à l'API, quelles routes sont concernées, et comment un frontend est censé piloter le flux de bout en bout. Pour les définitions des tables `User` / `OAuthUser`, voir la section Base de données.

Vue d'ensemble

Un utilisateur peut soit s'inscrire avec un mot de passe local (`/api/users/register/`), soit se connecter via un fournisseur d'identité externe. Chaque identité externe est stockée sous forme de ligne dans `OAuth_user`, liée à une ligne `user` - de sorte qu'un même compte puisse être atteint via plusieurs fournisseurs au fil du temps, ou créé de zéro dès la première connexion avec l'un d'eux.

`OAuth_user.provider` est une chaîne libre, pas une énumération : seul `"microsoft"` est implémenté aujourd'hui, mais rien dans le schéma ne restreint le champ à cette valeur.

Un utilisateur peut aussi **lier** une identité Microsoft à un compte qu'il possède déjà - typiquement un compte créé avec un mot de passe local - après coup, depuis la page de paramètres, sans avoir à se déconnecter au préalable. C'est une route distincte de celle de connexion ; voir Lier Microsoft à un compte existant ci-dessous.

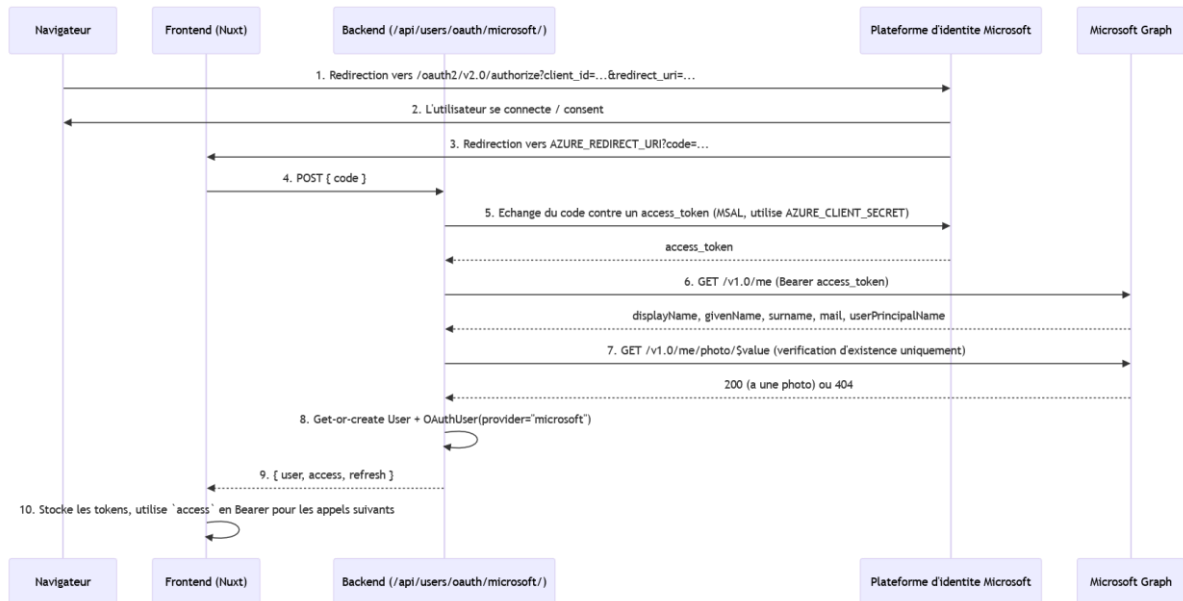
Comme le secret client confidentiel (`AZURE_CLIENT_SECRET`) ne doit jamais atteindre le navigateur, **c'est le backend - et non le frontend – qui effectue l'échange du code d'autorisation avec Microsoft**. Le frontend n'a que trois tâches : envoyer l'utilisateur vers Microsoft, récupérer le `code` renvoyé par Microsoft, et transmettre ce `code` au backend.

Routes concernées par OAuth

Méthode	Route	Auth	But
POST	<code>/api/users/oauth/microsoft/</code>	<code>AllowAny</code>	Échange un <code>code</code> d'autorisation Microsoft contre des tokens ; crée ou relie l'utilisateur (connexion/inscription)
POST	<code>/api/users/oauth/microsoft/link/</code>	<code>IsAuthenticated</code>	Échange un <code>code</code> et rattache l'identité Microsoft au compte déjà authentifié de l'appelant
POST	<code>/api/users/token/refresh/</code>	<code>AllowAny</code>	Rafraîchit un access token expiré (partagé avec la connexion par mot de passe)
GET	<code>/api/users/me/</code>	<code>IsAuthenticated</code>	Récupère le profil du propriétaire de l'access token courant

`/api/users/oauth/microsoft/` et `/api/users/oauth/microsoft/link/` sont les deux seules routes spécifiques à OAuth. Tout ce qui suit (rafraîchir les tokens, appeler `/me/`, etc.) est indiscernable d'une connexion classique par mot de passe - chaque flux converge vers la même paire de JWT (la liaison n'en émet même pas de nouvelle - voir plus bas).

Fonctionnement du flux Microsoft



Étape par étape, côté backend (backend/users/oauth_microsoft.py) :

1. `exchange_code_for_token(code)` construit une `ConfidentialClientApplication` MSAL à partir de `AZURE_CLIENT_ID` / `AZURE_CLIENT_SECRET` / `AZURE_AUTHORITY` et appelle `acquire_token_by_authorization_code(code, scopes=["User.Read"], redirect_uri=AZURE_REDIRECT_URI)`. Le `redirect_uri` passé ici doit correspondre exactement à celui utilisé pour obtenir le code à l'étape 1 du diagramme - sinon Microsoft rejette l'échange.

2. `fetch_microsoft_profile(access_token)` appelle l'endpoint `/me` de Graph pour récupérer `mail` / `userPrincipalName`, `givenName`, `surname`.

3. `has_microsoft_photo(access_token)` sonde `/me/photo/$value` pour vérifier si le compte a une photo, sans la télécharger.

4. `get_or_create_user_from_microsoft(profile, access_token)` fait correspondre un `User` existant par email, ou en crée un, puis fait un `update_or_create` sur la ligne `OAuthUser(provider="microsoft")` liée.

5. La vue (`MicrosoftOAuthView`) émet la même paire de JWT que `RegisterView` / `LoginView` et renvoie `{ user, access, refresh }`.

Si le code est invalide/expiré, ou si le compte Microsoft n'a pas d'email, la vue renvoie `400 Bad Request` avant toute écriture en base de données.

Lier Microsoft à un compte existant

En plus de se connecter, un utilisateur **déjà authentifié** peut rattacher son identité Microsoft au compte même sur lequel il est connecté – par exemple quelqu'un qui s'est inscrit avec un mot de passe local et veut maintenant que « Se connecter avec Microsoft » fonctionne aussi. C'est l'action « Lier mon compte Microsoft » de la page de paramètres, adossée à une route séparée, `POST /api/users/oauth/microsoft/link/` (`LinkMicrosoftOAuthView`), et à une fonction de service séparée, `link_microsoft_account()` dans `backend/users/oauth_microsoft.py`.

Elle réutilise exactement la même mécanique d'échange de code/récupération de profil que le flux de connexion (`exchange_code_for_token`, `fetch_microsoft_profile`, `has_microsoft_photo`), mais tout ce qui suit diffère :

	Connexion (<code>/oauth/microsoft/</code>)	Liaison (<code>/oauth/microsoft/link/</code>)
Auth requise	<code>AllowAny</code>	<code>IsAuthenticated</code>
Quel utilisateur	Apparié/créé par l'email du profil	Toujours <code>request.user</code> - jamais apparié/créé par email
En cas de succès	Émet une nouvelle paire de JWT	N'émet aucun token - la session existante de l'appelant continue de fonctionner telle quelle
Effet de bord sur le compte	Aucun au-delà de sa création	<code>user.email</code> est écrasé par l'email du profil Microsoft, et <code>user.set_unusable_password()</code> est appelé

Cette dernière ligne est importante : **la liaison est à sens unique et destructive pour le mot de passe local**. Une fois lié, le compte ne peut plus se connecter que via Microsoft - le mot de passe local cesse de fonctionner entièrement, il n'est pas conservé comme solution de repli. La fenêtre de confirmation du frontend (`pages/settings.vue`) l'indique explicitement avant de démarrer la redirection.

Comme la liaison réutilise le même enregistrement d'app Azure et le même redirect URI que la connexion (il n'y en a pas de second à configurer), le frontend a besoin d'un moyen de distinguer une tentative de connexion d'une tentative de liaison sur la page de callback partagée. Il fait cela avec le paramètre OAuth `state`, que Microsoft renvoie tel quel :

1. `useOAuth().startOAuth("microsoft", mode)(frontend/app/composables/useOAuth.ts)` ajoute `state=login` ou `state=link` à l'URL d'autorisation selon l'action qui l'a déclenché - la page de paramètres l'appelle avec `"link"`, partout ailleurs c'est `"login"` par défaut.

2. `finishOAuth("microsoft")`, appelé depuis la page de callback partagée (`pages/connect/microsoft/callback.vue`), relit `route.query.state` et fait un branchement : `"link"` envoie un `POST` à `/oauth/microsoft/link/` et appelle `updateUser()` (rien à stocker) ;

toute autre valeur envoie un POST à `/oauth/microsoft/` et appelle `setSession()` comme d'habitude.

3. En cas de succès, la page de callback redirige vers `/settings?linked=microsoft` (une connexion redirige elle vers `/home`) ; le paramètre `linked=microsoft` est ce qui déclenche le toast « Compte Microsoft lié avec succès. » sur `pages/settings.vue`.

Échoue avec 400 Bad Request si `code` est manquant, si l'échange avec Microsoft échoue de la même façon que pour la connexion, si le profil Graph n'a pas d'email, **ou si cet email appartient déjà à un compte différent**

`(User.objects.exclude(pk=user.pk).filter(email__iexact=email).exists())` - la liaison ne peut pas servir à s'approprier silencieusement le compte de quelqu'un d'autre.

Variables d'environnement

Définies dans le `.env` à la racine du dépôt (lu par `config/settings.py` de la même manière que `DATABASE_*` ; injecté via `env_file` dans `docker-compose.dev.yml` à l'intérieur des conteneurs) :

Variable	Signification
<code>AZURE_CLIENT_ID</code>	ID d'application (client) de l'enregistrement d'app Azure AD
<code>AZURE_TENANT_ID</code>	ID du tenant Azure AD dans lequel l'app est enregistrée
<code>AZURE_CLIENT_SECRET</code>	Secret client confidentiel - backend uniquement , jamais envoyé au frontend
<code>AZURE_REDIRECT_URI</code>	Doit correspondre à un redirect URI enregistré sur l'app Azure (actuellement la page <code>/connect/microsoft/callback</code> du frontend)
<code>AZURE_AUTHORITY</code>	Autorité de la plateforme d'identité Microsoft, ex. <code>https://login.microsoftonline.com/common/v2.0</code>

Les cinq variables ont des valeurs par défaut sûres (chaîne vide) dans `settings.py`, de sorte que l'application démarre même sans être configurée ; la route OAuth elle-même échouera (400/500) tant qu'elles ne sont pas renseignées.

Ajouter un autre fournisseur

Comme `OAuthUser.provider` est une simple chaîne, ajouter Google/GitHub/etc. plus tard ne nécessite pas de changement de schéma - seulement :

1. Un nouveau module de service (`users/oauth_<provider>.py`) reproduisant `oauth_microsoft.py` : échanger le code/token émis par le fournisseur, récupérer son endpoint de profil, appeler `get_or_create_user_from_<provider>` avec `provider="<provider>"`.

2. Une nouvelle vue + route sous `/api/users/oauth/<provider>/`.

3. De nouveaux réglages spécifiques au fournisseur (id/secret client/redirect URI).

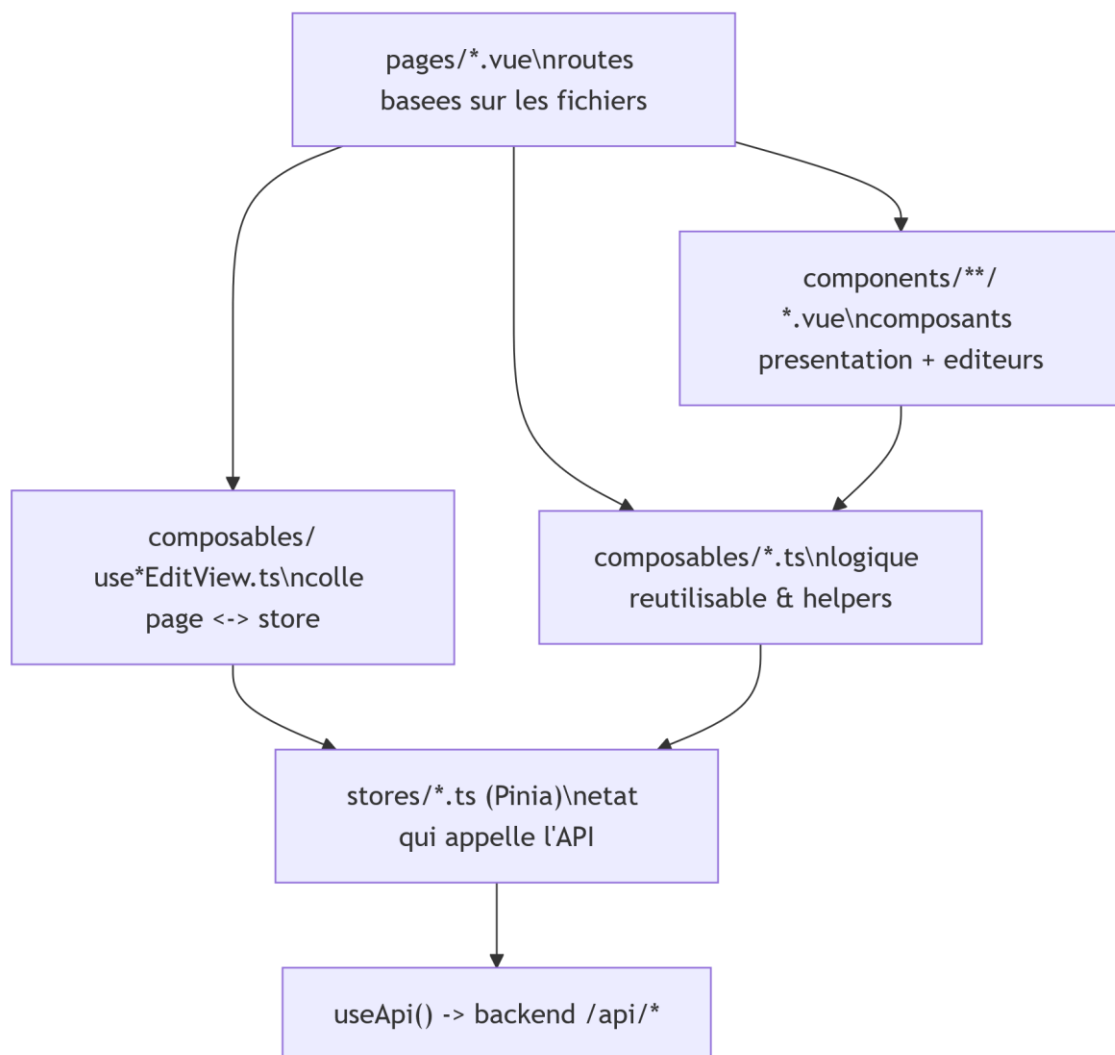
La logique d'appariement par email peut être extraite de `get_or_create_user_from_microsoft` si un second fournisseur est ajouté, afin que les deux partagent le même comportement « apparier par email, sinon créer ».

Frontend

Cette partie est la référence « quel fichier toucher pour X » du frontend Nuxt 4 / Vue 3 ([frontend/app/](#)). Il est découpé comme demandé, dans l'ordre des dépendances : **composants** (ce qui s'affiche), **stores** (ce qui appelle l'API), **composables** (la logique qui relie les deux), puis **pages principales** (ce qui utilise tout ça). Chaque nom est relié par un lien interne vers sa propre définition ailleurs dans ce document, pour passer directement d'une page à l'action de store qu'elle déclenche.

Pour la vue système (comment le frontend parle au backend, dev vs prod, etc.), voir « Architecture du projet ».

Vue d'ensemble



La plupart des pages sont fines : une page sous `recipes/`, `cookbooks/` ou `planning/` se contente en général d'afficher un composant `*Editor.vue` et laisse un composable `use*EditView.ts` correspondant (ex. `useRecipesEditView`) gérer l'état du formulaire, la validation et les appels au store. Les pages de lecture/liste appellent un composable de domaine (ex. `useRecipes`) qui encapsule le store correspondant.

Composants

Regroupés par sous-dossier de `frontend/app/components/`. La ligne **Dépend de** sous chaque tableau liste les composables/stores/composants enfants utilisés directement, reliés à leur propre section plus bas - c'est le moyen le plus rapide de savoir ce qui casse si on change le format de retour d'un composable.

Le dossier `icons/` (~35 SVG triviaux à un seul glyphe) est condensé dans une liste unique à la fin plutôt qu'une ligne par icône.

Branding & boutons

Composant	Objectif	Props	Emits
<code>AppLogo.vue</code>	Logo SUPMEAL avec un slogan/légende optionnel en dessous	<code>tagline?: string; size?: "sm" "md" "lg" (défaut "md")</code>	-
<code>button s/AppButton.vue</code>	Bouton/lien générique stylé - rend un <code><button></code> ou un <code>NuxtLink</code> selon que <code>to</code> est défini	<code>variant?: "primary" "secondary" "destructive" "ghost" "oauth" (défaut "primary"); size?: "sm" "md" "lg"; type?: "button" "submit" "reset"; disabled?: boolean; block?: boolean; to?: string</code>	-
<code>buttons/OAuthButtons.vue</code>	Boutons de connexion Google (désactivé, « bientôt ») et Microsoft OAuth ; démarre le flux Microsoft au clic	-	-

- `OAuthButtons.vue` - Composables : [useOAuth](#). Enfants : [AppButton](#), icônes.

Commun

Composant	Objectif	Props	Emits
<code>common/ConfirmModal.vue</code>	Modale de confirmation générique (téléportée dans le body); peut exiger un mot tapé et/ou une case à cocher	<code>open: boolean; title: string; message: string; confirmLabel?: string (défaut "Confirmer"); cancelLabel?: string (défaut "Annuler"); variant?: "default" "destructive"; typedWord?: string; checkboxLabel?: string</code>	<code>close: []; confirm: []</code>
<code>common/Pagination.vue</code>	Pagination numérotée, boutons préc/suiv + fenêtre de numéros (max 5)	<code>currentPage: number; totalPages: number</code>	<code>update:currentPage: [page: number]</code>
<code>common/ToastContainer.vue</code>	Affiche les notifications toast globales (succès/avertissement/erreur), téléportées dans le body	-	-

- `ToastContainer.vue` - Stores : [useToastStore](#). Rendu une seule fois, depuis `layouts/app.vue`.
- Utilisé par la plupart des composants d'édition/liste pour la confirmation d'action destructive ([DeleteCookbookModal](#), [DeletePlanningModal](#), [DeleteRecipeModal](#)) sont des enveloppes fines du même patron que `ConfirmModal`, avec un texte propre à leur domaine).

Cookbook

Composant	Objectif	Props	Emits
<code>cookbook/AssignCookbookMenu.vue</code>	Liste déroulante des cookbooks dont l'utilisateur est créateur, pour réassigner une recette/planning vers un autre	<code>currentCookbookId?: number null</code>	<code>select: [cookbook: { id: number; name: string }]</code>

Composant	Objectif	Props	Emits
<code></code> <code>>cookbook/CookbookCard.vue</code>	Carte de grille pour un cookbook personnel : icône, nom, compteurs recettes/plannings, date de mäj, menu « ... » (voir/éditer/supprimer)	<code>cookbook: Cookbook; to: string; showMenu?: boolean (défaut true)</code>	-
<code></code> <code>cookbook/CookbookDiscussionSidebar.vue</code>	Barre latérale droite (rendue depuis <code>layouts/app.vue</code>) avec une discussion par cookbook : sélecteur de canal (général/recette/planning), messages, envoi/suppression	<code>cookbookId: number; defaultChannel: CookbookDiscussionDefault</code>	-
<code></code> <code>cookbook/CookbookEditor.vue</code>	Formulaire de création/édition d'un cookbook : nom, upload d'icône, sauvegarde/suppression, retour	<code>mode: "create" "edit"; cookbookId?: number</code>	-
<code></code> <code>cookbook/CookbookMembersPanel.vue</code>	Liste le créateur + les membres partagés ; le propriétaire peut inviter par email, changer un rôle, ou retirer l'accès	<code>cookbookId: number; creator; sharedWith: SharedUserCookbook[]; isOwner: boolean</code>	-
<code></code> <code>cookbook/DeleteCookbookModal.vue</code>	Modale de confirmation destructive pour supprimer un cookbook ; peut conserver ses recettes/plannings en tant que personnels, verrouillée derrière la saisie de « suppression »	<code>open: boolean; cookbookName: string; recipes?: {id,title}[]; plannings?: {id,name}[]</code>	<code>close: [];</code> <code>confirm: [options: {keepRecipes: boolean; keepPlannings: boolean}]</code>
<code></code> <code>cookbook/SharedCookbookCard.vue</code>	Carte-ligne compacte pour un cookbook partagé avec l'utilisateur courant : icône, nom, propriétaire, activité, badge de rôle	<code>cookbook: Cookbook; role: Exclude<CookbookRole, "admin">; to: string; activityAt?: string</code>	-

- AssignCookbookMenu.vue - Stores : [useCookbookStore](#). Composables : [useAuth](#), hasCookbookRank ([useCookbooks](#)).
- CookbookCard.vue - Composables : [useRecipes](#) (relativeTime), [useCookbooksEditView](#) (deleteCookbookWithOptions). Stores : [useToastStore](#). Enfants : [DeleteCookbookModal](#).
- CookbookDiscussionSidebar.vue - Appelle [useApi](#) directement, pas via un store, sur les routes de messagerie imbriquées (/cookbooks/{id}/messages/, .../recipes/{id}/messages/, .../plannings/{id}/messages/) - voir [stores/cookbooks/useMessageStore.ts](#) pour comprendre pourquoi. Utilise aussi [useAuth](#), [useCookbooks](#). Stores : [useCookbookStore](#), [useToastStore](#).
- CookbookEditor.vue - Composables : [useCookbooksEditView](#) (useCookbookEditForm), [useGoBack](#). Enfants : [DeleteCookbookModal](#).
- CookbookMembersPanel.vue - Stores : [useCookbookStore](#), [useToastStore](#). Composables : [useCookbooks](#) (COOKBOOK_ROLE_LABELS).
- SharedCookbookCard.vue - Composables : [useCookbooks](#) (COOKBOOK_ROLE_LABELS), [useRecipes](#) (relativeTime).

Utilisé par : [pages/cookbooks/index.vue](#), [.../new.vue](#), [.../\[id\]/view.vue](#), [.../\[id\]/edit.vue](#), ainsi que CookbookCard/SharedCookbookCard sur [pages/home.vue](#) et [pages/sharedwithme.vue](#).

Formulaires

Composant	Objectif	Props	Emits
<code>forms/AppInput.vue</code>	Champ texte avec label, slot d'icône de tête optionnel, texte d'erreur/aide, bascule intégrée afficher/masquer pour <code>type="password"</code>	<code>modelValue?: string;</code> <code>id: string;</code> <code>label?: string;</code> <code>type?: string</code> (défaut <code>"text"</code>); <code>placeholder?: string;</code> <code>autocomplete?: string;</code> <code>hint?: string;</code> <code>error?: string</code>	<code>update:modelValue:</code> <code>[value: string]</code>

- Utilisé sur tous les formulaires d'auth/paramètres : [pages/login.vue](#), [register.vue](#), [settings.vue](#).

Import / export

Composant	Objectif	Props	Emits
<code>import_export/ExportSelectionList.vue</code>	Liste à cocher multi-sélection, recherchable et paginée (recettes/cookbooks/plannings) avec « tout sélectionner (filtré) » + bouton export	<code>items:</code> <code>SelectableItem[];</code> <code>loading?: boolean;</code> <code>exporting?: boolean;</code> <code>emptyLabel?: string;</code> <code>searchPlaceholder?: string</code>	<code>export:</code> <code>[ids: number[]]</code>
<code>import_export/ImportDropzone.vue</code>	Zone de glisser-déposer (ou parcourir) pour un fichier .json unique, avec aperçu/effacement et bouton d'import ; expose <code>clearFile()</code>	<code>label:</code> <code>string;</code> <code>busy?: boolean</code>	<code>import:</code> <code>[file: File]</code>

- `ExportSelectionList.vue` - Utilise le type `SelectableItem` de [useImportExport](#). Enfants : [Pagination](#).
- `ImportDropzone.vue` - Composables : [useImportExport](#) (`isJsonFile`), `VueUse` `useDropZone`. Stores : [useToastStore](#).
- Les deux sont utilisés exclusivement par [pages/import_export.vue](#).

Layout

Composant	Objectif	Props	Emits
<code>layout/AppSidebar.vue</code>	Barre latérale gauche fixe : avatar, logo, nav principale (accueil/recettes/cookbooks/partagés/planning/recherche/import-export), nom d'utilisateur, paramètres, déconnexion	-	-

- Composables : [useAuth](#). Rendu une seule fois, depuis `layouts/app.vue` (voir [Pages principales](#) pour les pages utilisant ce layout).

Planning

Composant	Objectif	Props	Emits
<code></code> <code>planning/DeletePlanningModal.vue</code>	Modale de confirmation destructive pour supprimer un planning, verrouillée derrière la saisie de « suppression »	<code>open: boolean;</code> <code>planningName: string</code>	<code>close: [];</code> <code>confirm: []</code>
<code></code> <code>planning/MealPlanEditor.vue</code>	Grille éditable de plan de repas (moment x plat x jour) ; chaque cellule est un RecipePicker	<code>type: PlanningType;</code> <code>slots: MealSlot[];</code> <code>recipes?: Recipe[]</code>	<code>update-slot: [key: string, pick: RecipePick]</code>
<code></code> <code>planning/PlanningCard.vue</code>	Carte de liste pour un planning : icône, nom, type/nb repas/date de mäj, menu « ... » (voir/éditer/réassigner cookbook/supprimer)	<code>planning: Planning;</code> <code>to: string;</code> <code>canEdit?/canManage?/canReassignCookbook?: boolean (défaut true);</code> <code>showMenu?: boolean</code>	-
<code></code> <code>planning/PlanningEditor.vue</code>	Formulaire de création/édition d'un planning : nom, icône, sélecteur de type (création uniquement), badge cookbook, grille de repas	<code>mode: "create" "edit";</code> <code>planningId?: number</code>	-
<code></code> <code>planning/PlanningMealsGrid.vue</code>	Grille de plan de repas en lecture seule, recettes planifiées sous forme de liens, pour visualiser un planning existant	<code>type: PlanningType;</code> <code>meals: PlanningMeal[]</code>	-
<code></code> <code>planning/RecipePicker.vue</code>	Autocomplétion de recette pour une cellule (v-model), utilisée dans MealPlanEditor ; recherche parmi les recettes de l'utilisateur, ou filtre une liste	<code>expanded?: boolean;</code> <code>recipes?: Recipe[];</code> <code>modèle :</code>	<code>update:modelValue (implicite)</code>

Composant	Objectif	Props	Emits
g/RecipePicker.vue	recipes fixe quand scopé à un cookbook	defineModel<RecipePicker>	

- MealPlanEditor.vue - Composables : [usePlanning](#) (DAYS_OF_WEEK/MEAL_MOMENTS/MEAL_COURSES/DAILY_PLANNING_DAY). Enfants : [RecipePicker](#).
- PlanningCard.vue - Stores : [usePlanningStore](#), [useToastStore](#). Composables : [useRecipes](#) (relativeTime), [usePlanning](#) (PLANNING_TYPE_LABELS). Enfants : [DeletePlanningModal](#), [AssignCookbookMenu](#).
- PlanningEditor.vue - Composables : [usePlanningEditView](#) (usePlanningEditForm), [usePlanning](#), [useGoBack](#). Enfants : [MealPlanEditor](#), [DeletePlanningModal](#).
- PlanningMealsGrid.vue - Composables : [usePlanning](#) (constantes + mealsByDayAndSlot).
- RecipePicker.vue - Composables : [useRecipes](#) (searchMyRecipes).

Utilisé par : [pages/planning/index.vue](#), new.vue, [id]/view.vue, [id]/edit.vue.

Recettes

Composant	Objectif	Props	Emits
recipes/DeleteRecipeModal.vue	Modale de confirmation destructive pour supprimer une recette ; avertit si utilisée dans des plannings, verrouillée derrière la saisie de « suppression »	open: boolean; recipeTitle: string; usedInPlannings?: PlanningUsage[]	close: []; confirm: []
recipes/IngredientRow.vue	Ligne d'ingrédient unique : nom avec autocomplétion sur le catalogue, quantité/unité/nb de personnes, upload d'image, avertissement de	isDuplicate?: boolean; excludeIngredientIds?: number[]; modèle : defineModel<IngredientLine>	update:modelValue (implicite); remove: []

Composant	Objectif	Props	Emits
	doublon, suppression		
<code>recipes/IngredientsPanel.vue</code>	Enveloppe une liste de IngredientRow , bouton « ajouter un ingrédient », champ portions par défaut	modèle : defineModel<IngredientLine[]>	update:modelValue (implicite)
<code>recipes/RecipeCard.vue</code>	Carte de grille pour une recette : image, titre, bascule tags, bascule favori, menu « ... » (voir/éditer/réassigner cookbook/supprimer)	recipe: Recipe; to: string; canEdit?/canManage?/canReassignCookbook?: boolean; showMenu?: boolean	-
<code>recipes/RecipeEditor.vue</code>	Formulaire de création/édition d'une recette : titre, image, source, durée de cuisson, étapes, ingrédients, tags, sauvegarde/suppression	mode: "create" "edit"; recipeId?: number	-
<code>recipes/StepEditor.vue</code>	Éditeur d'une étape de recette : éditeur WYSIWYG Tiptap (gras/italique/liste à puces) sérialisé en markdown, sélecteur de type (préparation/cuisson), durée, réordonner/supprimer	index: number; total: number; modèle : defineModel<StepLine>	update:modelValue (implicite); remove: []; move-up: []; move-down: []
<code>recipes/TagsPanel.vue</code>	Ajouter/retirer les tags d'une recette : filtre par type (repas/régime), recherche-et-choix	modèle : defineModel<TagLine[]>	update:modelValue (implicite)

Composant	Objectif	Props	Emits
	parmi les tags existants, ou création d'un nouveau		

- `DeleteRecipeModal.vue` - pas de dépendance notable en dehors des icônes.
- `IngredientRow.vue` - Stores : [useRecipeStore](#). Composables : [useRecipes](#) (`fileToDataUrl`).
- `IngredientsPanel.vue` - Composables : [useRecipes](#) (`emptyIngredientLine`). Enfants : [IngredientRow](#).
- `RecipeCard.vue` - Stores : [useRecipeStore](#), [useToastStore](#). Composables : [useRecipes](#), [useCookbooks](#) (`useCookbookName`). Enfants : [DeleteRecipeModal](#), [AssignCookbookMenu](#).
- `RecipeEditor.vue` - Composables : [useRecipesEditView](#) (`useRecipeEditForm`), [useGoBack](#). Enfants : [StepEditor](#), [IngredientsPanel](#), [TagsPanel](#), [DeleteRecipeModal](#).
- `StepEditor.vue` - Tiers : `@tiptap/vue-3`, `@tiptap/starter-kit`, `@tiptap/extension-placeholder`, `tiptap-markdown`.
- `TagsPanel.vue` - Stores : [useRecipeStore](#). Composables : [useRecipes](#) (`uid`, `capitalizeFirst`).

Utilisé par : [pages/new.vue](#), [pages/recipes/index.vue](#), `[id]/view.vue`, `[id]/edit.vue`.

Recherche

Composant	Objectif	Props	Emits
<code>search/CatalogMultiSelect.vue</code>	Multi-sélection déroulante sur un catalogue recherché côté serveur (tags/ingrédients), liée à une chaîne séparée par virgules ; les éléments sélectionnés s'affichent en puces amovibles	<pre>label: string; searchPlaceholder?: string; emptyLabel?: string; fetchOptions: (search: string) => Promise<{id,name,image?}[]>; capitalizeLabels?: boolean; modèle : defineModel<string></pre>	<code>update:modelValue</code> (implicite)
<code>search/CookbookSearchResults.vue</code>	Grille de CookbookCard (squelettes de chargement / état vide / résultats)	<pre>cookbooks: Cookbook[]; isLoading: boolean</pre>	-
<code>search/PlanningSearchResults.vue</code>	Grille de PlanningCard (squelettes de chargement / état vide / résultats)	<pre>plannings: Planning[]; isLoading: boolean</pre>	-
<code>search/RecipeSearchResults.vue</code>	Grille de RecipeCard (squelettes de chargement / état vide / résultats)	<pre>recipes: Recipe[]; isLoading: boolean</pre>	-
<code>search/SearchFilters.vue</code>	Panneau de filtres dont les champs changent selon type (recettes/plannings/cookbooks) : nom, ingrédients/tags, scope cookbook/planning, favoris, plages de temps préparation/cuisson, boutons réinitialiser/rechercher	<pre>type: SearchType; showCookbookScope?: boolean; modèles : recipeFilters, planningFilters, cookbookFilters</pre>	<pre>search: []; reset: []</pre>

- `CatalogMultiSelect.vue` - Composables : [useRecipes](#) (`capitalizeFirst`), `VueUse` `onClickOutside`.
- `SearchFilters.vue` - Stores : [useRecipeStore](#). Composables : [usePlanning](#) (`PLANNING_TYPE_LABELS`). Enfants : [CatalogMultiSelect](#).

Utilisé par : [pages/search.vue](#) (les quatre) et [pages/cookbooks/\[id\]/view.vue](#) (`SearchFilters` seul, filtrant les onglets propres à un cookbook).

Paramètres

Composant	Objectif
<code>settings/CuisinePreferencesPanel.vue</code>	Gère les préférences de cuisine/tags préférés de l'utilisateur : préférences actuelles en puces amovibles, recherche-et-bascule sur le catalogue de tags

- Stores : [useRecipeStore](#). Composables : [useCuisinePreferences](#), [useRecipes](#) (`capitalizeFirst`).
- Utilisé par [pages/settings.vue](#).

Icônes

Les ~35 fichiers sous `components/icons/` sont des SVG triviaux à un seul glyphe partageant une seule prop : `size?: "xs" | "sm" | "md" | "lg"` (défaut "md").

Stores

Chaque store Pinia, et **chaque endpoint backend qu'il appelle** - c'est la liste à vérifier avant de supposer « il y a sûrement déjà une méthode de store pour ça », ou avant d'ajouter un nouvel appel API (ajoutez-le ici, n'appellez pas `useApi()` depuis un composant - voir l'unique exception volontaire dans [CookbookDiscussionSidebar](#)). Tous les endpoints sont relatifs à `/api/`.

`useCookbookStore`

Possède l'état liste/détail des cookbooks, le partage, et un cache de noms. État : `cookbooks`, `currentCookbook`, `cookbookNames` (id -> nom, cache), `pagination`, `loading`, `error`.

Action	Méthode & endpoint	Objectif
<code>fetchCookbooks(params)</code>	GET <code>/cookbooks/</code>	Lister/paginer les cookbooks (<code>name</code> , <code>shared_with_me</code> , <code>page</code> , <code>page_size</code>)
<code>fetchCookbook(id)</code>	GET <code>/cookbooks/{id}/</code>	Charger le détail complet d'un cookbook
<code>createCookbook(payload)</code>	POST <code>/cookbooks/</code>	Créer un cookbook
<code>updateCookbook(id, payload)</code>	PATCH <code>/cookbooks/{id}/</code>	Mettre à jour nom/icône
<code>deleteCookbook(id)</code>	DELETE <code>/cookbooks/{id}/</code>	Supprimer un cookbook
<code>shareCookbook(id, shares)</code>	POST <code>/cookbooks/{id}/share/</code>	Ajouter/mettre à jour des membres partagés et leurs rôles
<code>unshareCookbook(id, userIds)</code>	POST <code>/cookbooks/{id}/unshare/</code>	Retirer des membres partagés
<code>fetchCookbookName(id)</code>	GET <code>/cookbooks/{id}/</code> (mis en cache)	Résoudre juste le nom d'un cookbook (labels type fil d'Ariane)

Utilisé via [useCookbooks](#) / [useCookbooksEditView](#), par [AssignCookbookMenu](#), [CookbookCard](#), [CookbookMembersPanel](#), [CookbookDiscussionSidebar](#), et les [pages cookbooks](#).

`useMessageStore` (vide)

`stores/cookbooks/useMessageStore.ts` est actuellement un **fichier de 0 octet** - non importé ni référencé nulle part. Il n'existe pas de store de messages dédié ; `CookbookDiscussionSidebar.vue` appelle `useApi` directement sur les endpoints `messaging` à la place. Si un store de messages est ajouté plus tard, c'est ce fichier qui doit l'accueillir.

`useImportExportStore`

Possède l'état des requêtes d'export/import JSON pour recettes et cookbooks.
État : `exportingRecipes`, `exportingCookbooks`, `importingRecipes`, `importingCookbooks`, `error`.

Action	Méthode & endpoint	Objectif
<code>exportRecipes(ids)</code>	GET <code>/recipes/export/</code>	Exporter tout ou partie des recettes personnelles en JSON
<code>exportCookbooks(ids)</code>	GET <code>/cookbooks/export/</code>	Exporter tout ou partie des cookbooks possédés en JSON
<code>importRecipes(payload)</code>	POST <code>/recipes/import/</code>	Importer une/des recette(s) depuis un JSON parsé
<code>importCookbooks(payload)</code>	POST <code>/cookbooks/import/</code>	Importer un/des cookbook(s) depuis un JSON parsé

Utilisé via `useImportExport`, par `ExportSelectionList`, `ImportDropzone` et `pages/import export.vue`.

`usePlanningStore`

Possède l'état liste/détail des plannings de repas. État : `plannings`, `currentPlanning`, `pagination`, `loading`, `error`.

Action	Méthode & endpoint	Objectif
<code>fetchPlannings(params)</code>	GET <code>/plannings/</code>	Lister/paginer les plannings (<code>name</code> , <code>type</code> , <code>cookbook</code> , <code>in_cookbook</code> , <code>shared_with_me</code> , <code>page</code> , <code>page_size</code>)
<code>fetchPlanning(id)</code>	GET <code>/plannings/{id}/</code>	Charger le détail complet d'un planning (grille de repas)

Action	Méthode & endpoint	Objectif
<code>createPlanning(payload)</code>	POST <code>/plannings/</code>	Créer un planning (avec ses créneaux de repas)
<code>updatePlanning(id, payload)</code>	PATCH <code>/plannings/{id}/</code>	Mettre à jour nom/icône/type/repas
<code>deletePlanning(id)</code>	DELETE <code>/plannings/{id}/</code>	Supprimer un planning

Utilisé via [usePlanning](#) / [usePlanningEditView](#), par [PlanningCard](#), [PlanningEditor](#) et les [pages planning](#).

useRecipeStore

Possède l'état liste/détail des recettes ainsi que les catalogues partagés de tags/ingrédients.
 État : `recipes`, `currentRecipe`, `tags`, `ingredients`, `pagination`, `loading`, `error`.

Action	Méthode & endpoint	Objectif
<code>fetchRecipes(params)</code>	GET <code>/recipes/</code>	Lister/paginer les recettes (<code>name</code> , <code>tags</code> , <code>ingredients</code> , <code>cookbook</code> , <code>in_cookbook</code> , <code>planning</code> , <code>in_planning</code> , <code>favorite</code> , <code>shared_with_me</code> , <code>page</code> , <code>page_size</code>)
<code>fetchRecipe(id)</code>	GET <code>/recipes/{id}/</code>	Charger le détail complet d'une recette
<code>createRecipe(payload)</code>	POST <code>/recipes/</code>	Créer une recette (ingrédients/tags/étapes)
<code>updateRecipe(id, payload)</code>	PATCH <code>/recipes/{id}/</code>	Mettre à jour une recette
<code>deleteRecipe(id)</code>	DELETE <code>/recipes/{id}/</code>	Supprimer une recette
<code>setFavorite(id, favorite)</code>	POST / DELETE <code>/recipes/{id}/favorite/</code>	Ajouter/retirer une recette des favoris
<code>fetchTags(search)</code>	GET <code>/tags/</code>	Récupérer/rechercher le catalogue de tags partagé

Action	Méthode & endpoint	Objectif
<code>fetchIngredients(search)</code>	GET <code>/ingredients/</code>	Récupérer/rechercher le catalogue d'ingrédients partagé

Utilisé via [useRecipes](#) / [useRecipesEditView](#), par quasiment tous les composants liés aux recettes ainsi que [CuisinePreferencesPanel](#), [SearchFilters](#), [TagsPanel](#), [IngredientRow](#).

`useToastStore`

Possède la file de notifications toast de l'app. **Aucun appel backend.** État : `toasts` (tableau de `{id, type, message}`). Actions : `push()`, `success()`, `warning()`, `error()`, `remove(id)` - purement client, auto-fermeture via `setTimeout`.

Consommé par [ToastContainer](#) (l'affiche) et par quasiment tous les stores/composables qui font une mutation (retour succès/erreur).

`useUserStore`

Possède les mutations de gestion de compte (paramètres). L'état de session/utilisateur courant vit lui dans [useAuth](#), pas ici. État : `loading`, `error`.

Action	Méthode & endpoint	Objectif
<code>updateUsername(id, username)</code>	PATCH <code>/users/{id}/</code>	Changer le nom d'utilisateur du compte
<code>changeEmail(newEmail, newPassword?)</code>	POST <code>/users/change-email/</code>	Changer l'email (les comptes OAuth définissent aussi un nouveau mot de passe local)
<code>linkMicrosoftAccount(code)</code>	POST <code>/users/oauth/microsoft/link/</code>	Lier un compte Microsoft à la session courante
<code>changePassword(currentPassword, newPassword)</code>	POST <code>/users/change-password/</code>	Changer le mot de passe du compte
<code>changeAvatar(dataUri)</code>	POST <code>/users/change-avatar/</code>	Mettre à jour l'icône/avatar de profil

Action	Méthode & endpoint	Objectif
<code>deleteAccount(id)</code>	DELETE <code>/users/{id}/</code>	Supprimer le compte de l'utilisateur

Chaque action de mutation appelle aussi `useAuth().updateUser(...)` pour synchroniser l'utilisateur en cache de la session. Utilisé via [useChangeEmail](#) / [useChangePassword](#), par [pages/settings.vue](#).

Composables

Logique réactive réutilisable sous `frontend/app/composables/`. Les composables `use*EditView.ts` sont la couche « colle de page » : chacun possède l'état d'un formulaire de création/édition et un composable de vue/détail correspondant, tous deux adossés à un store.

Composable	Objectif	Retourne	Dépend de
<code>useAPI.ts (useApi)</code>	Client HTTP central pour le backend Django REST. URL de base depuis <code>runtimeConfig.public.apiUrl</code> ; injecte <code>Authorization: Bearer <access></code> depuis <code>useToken</code> ; sur un 401, vide la session et redirige vers <code>/login</code> côté client	<code>get/post/put/patch/delete, extractMembers, extractData</code>	useToken
<code>useAuth.ts</code>	État de session/auth : login, register, logout, refresh de token, réhydratation au démarrage de l'app	<code>user, isAuthenticated, register(), login(), logout(), initAuth(), refreshSession(), setSession(), updateUser(), clearSession()</code>	useApi , useToken
<code>useToken.ts</code>	Stockage brut des JWT access/refresh (<code>useState Nuxt, SSR-safe</code>)	<code>access, refresh (refs)</code>	- (fondamental)
<code>auth.ts</code>	Schémas de validation Zod pour les formulaires login/register	<code>loginSchema, registerSchema</code>	-
<code>useCookbooks.ts</code>	Aides rôle/permission + raccourcis de récupération de listes pour les cookbooks	<code>COOKBOOK_ROLE_RANK, COOKBOOK_ROLE_LABELS, getCookbookRole(), hasCookbookRank(), getCookbookLastActivity(), useCookbookRoleFor(), useCookbooks() -> {store, fetchMyCookbooks, fetchRecentCookbooks,</code>	useCookbookStore , useAuth

Composable	Objectif	Retourne	Dépend de
		<code>fetchSharedCookbooks</code> <code>}, useCookbookName()</code>	
<code>useCookbooksEditView.ts</code>	Logique de page pour la création/édition de cookbook + vue/détail (dont suppression en cascade)	<code>deleteCookbookWithOptions()</code> , <code>useCookbookEditForm(props)</code> , <code>useCookbookView(cookbookId)</code>	useCookbookStore , useRecipeStore , usePlanningStore , useToastStore , useAuth , useRecipes , useCookbooks , useCookbookDiscussionContext
<code>useCookbookDiscussionContext.ts</code>	État partagé au niveau layout : quel cookbook afficher dans la barre de discussion (+ canal par défaut) à côté de la page courante ; se réinitialise automatiquement à chaque navigation via une garde de routeur	<code>useCookbookDiscussionContext()</code> -> <code>Ref<CookbookDiscussionContext null></code>	- (utilisé par <code>layouts/app.vue</code> + les pages scopées à un cookbook)
<code>useCuisinePreferences.ts</code>	Préférences de cuisine/tags préférés par utilisateur, stockées uniquement en localStorage (aucun champ backend)	<code>preferences</code> , <code>isPreferred()</code> , <code>togglePreference()</code>	useAuth (pour clé par id utilisateur)
<code>useGoBack.ts</code>	Retourner dans l'historique de l'app, ou se replier sur une route donnée	<code>useGoBack(fallback)</code>	Routeur Nuxt
<code>useImportExport.ts</code>	Orchestration UI de l'export/import JSON (validation de fichier, téléchargement, toasts)	<code>isJsonFile()</code> , <code>readJsonFile()</code> , <code>downloadJson()</code> , <code>useImportExport()</code> -> <code>{store,</code> <code>exportRecipes,</code> <code>exportCookbooks,</code> <code>importRecipes,</code> <code>importCookbooks}</code>	useImportExportStore , useToastStore

Composable	Objectif	Retourne	Dépend de
<code>useOAuth.ts</code>	Flux de connexion/liaison OAuth Microsoft (redirection + échange au retour)	<code>startOAuth(provider, mode), finishOAuth(provider)</code>	useApi , useAuth
<code>usePagination.ts (fetchAllPages)</code>	Parcourt toutes les pages d'un endpoint paginé et concatène les résultats	<code>fetchAllPages<T>(fetchPage)</code>	-
<code>usePlanning.ts</code>	Constantes/aides du domaine planning (jours, moments/plats de repas, génération de grille de créneaux) + raccourcis de récupération de listes	<code>DAYS_OF_WEEK, MEAL_MOMENTS, MEAL_COURSES, PLANNING_TYPE_LABELS, DAILY_PLANNING_DAY, slotUid(), generateEmptySlots(), planningToMealSlots(), mealsByDayAndSlot(), usePlanning() -> {store, fetchMyPlannings, fetchRecentPlannings}</code>	usePlanningStore
<code>usePlanningEditView.ts</code>	Logique de page pour la création/édition de planning + vue/détail	<code>usePlanningEditForm(props), usePlanningView(planningId)</code>	usePlanningStore , useToastStore , useAuth , useRecipes , usePlanning , useCookbookStore , useCookbooks , useCookbookDiscussionContext
<code>useRecipes.ts</code>	Aides du domaine recette (constructeurs ingrédient/tag/étape, validation d'image, formatage de durée, rendu	Nombreux helpers purs (<code>emptyIngredientLine</code> , <code>minutesToDury/duryToMinutes</code> , <code>fileToDataUrl</code> , <code>isAllowedImageFile</code> ,	useRecipeStore

Composable	Objectif	Retourne	Dépend de
	markdown) + raccourcis de récupération de listes	<pre>relativeTime, sumStepMinutes, formatCookingDuration, formatNumber, capitalizeFirst, renderStepMarkdown, recipeToIngredientLines/TagLines/StepLines, sortFavoritesFirst), useRecipes() -> {store, fetchMyRecipes, fetchRecentRecipes, searchMyRecipes, toggleFavorite}</pre>	
<pre> useRecipesEditView.ts</pre>	Logique de page pour la création/édition de recette + vue/détail (dont mise à l'échelle du nombre de portions)	<pre>useRecipeEditForm(props), useRecipeView(recipeId)</pre>	useRecipeStore , useToastStore , useAuth , useCookbooks , useCookbookDiscussionContext , useRecipes
<pre> useSearch.ts</pre>	Formes d'état de recherche/filtre + filtrage client-side pur (utilisé dans les onglets déjà chargés d'un cookbook)	Types <pre>(RecipeFilterState, PlanningFilterState, CookbookFilterState), createRecipeFilters/ createPlanningFilters/ createCookbookFilters, filterRecipesLocally(), filterPlanningsLocally()</pre>	useRecipes (sumStepMinutes)
<pre> managingUser/ changeemail.ts (useChangeEmail)</pre>	Logique de formulaire pour « changer l'email » dans les paramètres (cas particulier compte OAuth nécessitant un nouveau mot de passe)	<pre>form, errors, isSubmitting, validate, submit()</pre>	<pre>useFormValidation, changeEmailSchema, useUserStore, useToastStore</pre>

Composable	Objectif	Retourne	Dépend de
<code>managingUser/changepassword.ts (useChangePassword)</code>	Logique de formulaire pour « changer le mot de passe » dans les paramètres	<code>form, errors, isSubmitting, submit()</code>	<code>useFormValidation,</code> <code>changePasswordSchema,</code> useUserStore , useToastStore
<code>managingUser/schemas.ts</code>	Schémas Zod pour les formulaires de changement de nom d'utilisateur/email/mot de passe	<code>changeUsernameSchema,</code> <code>changeEmailSchema (requirePassword),</code> <code>changePasswordSchema</code>	-
<code>managingUser/useZodForms.ts (useFormValidation)</code>	Aide générique de formulaire réactif + validation Zod	<code>form, errors, validate()</code>	-

Utilisé par : voir les puces **Dépend de** sous chaque [composant](#) et la colonne **Composables/Stores utilisés** de [Pages principales](#).

Pages principales

Routes basées sur les fichiers sous `frontend/app/pages/. layouts/app.vue` est la coquille authentifiée (rend [AppSidebar](#) + [ToastContainer](#), plus [CookbookDiscussionSidebar](#) quand [useCookbookDiscussionContext](#) est défini) ; `layouts/empty.vue` est nu ; `layout: false` signifie aucun habillage. Chaque route est protégée par `middleware/auth.global.ts`, sauf les routes publiques (`/public_home`, `/login`, `/register`, `/logout`, `/connect/microsoft/callback`).

Auth / public

Page (route)	Objectif	Composants utilisés	Composables/Stores utilisés	Layout
public_home.vue (/public_home)	Page d'accueil pour les visiteurs non connectés	AppLogo , AppButton	-	false
login.vue (/login)	Connexion email/mot de passe + OAuth	AppLogo , AppInput , AppButton , OAuthButtons	useAuth (login), loginSchema (auth.ts), useFormValidation	false
register.vue (/register)	Inscription (email/mot de passe + OAuth)	AppLogo , AppInput , AppButton , OAuthButtons	useAuth (register), registerSchema (auth.ts), useFormValidation	false
logout.vue (/logout)	Déconnexion au montage, écran de confirmation	AppLogo , AppButton	useAuth (logout)	false
connect/microsoft/callback.vue (/connect/microsoft/callback)	Gère la redirection OAuth Microsoft (connexion ou liaison de compte)	AppLogo	useOAuth (finishOAuth)	empty

Recettes

Page (route)	Objectif	Composants utilisés	Composables/Stores utilisés	Layout
<code>new.vue (/new)</code>	Créer une nouvelle recette	RecipeEditor (mode="create")	useRecipesEditView , useGoBack ; useRecipeStore , useToastStore	app
<code>recipes/index.vue (/recipes)</code>	Lister/rechercher les recettes personnelles, paginé	AppButton , RecipeCard , Pagination	useRecipes (fetchMyRecipes, sortFavoritesFirst)	app
<code>recipes/[id]/view.vue (/recipes/:id/view)</code>	Détail de recette - ingrédients mis à l'échelle, étapes, favori/suppression/édition	AppButton , DeleteRecipeModal	useRecipesEditView (useRecipeView), useRecipes , useGoBack	app
<code>recipes/[id]/edit.vue (/recipes/:id/edit)</code>	Éditer une recette existante	RecipeEditor (mode="edit")	useRecipesEditView , useGoBack	app

Cookbooks

Page (route)	Objectif	Composants utilisés	Composables/Stores utilisés	Layout
<code>cookbooks/index.vue</code> (<code>/cookbooks</code>)	Lister/rechercher les cookbooks possédés, paginé	AppButton , CookbookCard , Pagination	useCookbooks (<code>fetchMyCookbooks</code>)	app
<code>cookbooks/new.vue</code> (<code>/cookbooks/new</code>)	Créer un nouveau cookbook	CookbookEditor (<code>mode="create"</code>)	useCookbooksEditView , useGoBack	app
<code>cookbooks/[id]/view.vue</code> (<code>/cookbooks/:id/view</code>)	Détail de cookbook - onglets recettes/planning/membres, filtre + pagination par onglet	AppButton , RecipeCard , PlanningCard , CookbookMembersPanel , SearchFilters , Pagination , DeleteCookbookModal	useCookbooksEditView (<code>useCookbookView</code>), useGoBack , useCookbooks , useSearch	app
<code>cookbooks/[id]/edit.vue</code> (<code>/cookbooks/:id/edit</code>)	Éditer un cookbook existant	CookbookEditor (<code>mode="edit"</code>)	useCookbooksEditView	app

Planning

Page (route)	Objectif	Composants utilisés	Composables/Stores utilisés	Layout
<code>planning/index.vue</code> (<code>/planning</code>)	Lister/rechercher les plannings personnels, paginé	AppButton , PlanningCard , Pagination	usePlanning (<code>fetchMyPlannings</code>)	app
<code>planning/new.vue</code> (<code>/planning/new</code>)	Créer un nouveau planning de repas	PlanningEditor (<code>mode="create"</code>)	usePlanningEditView , useGoBack	app
<code>planning/[id]/view.vue</code> (<code>/planning/:id/view</code>)	Détail de planning - grille de repas, édition/suppression	AppButton , PlanningMealsGrid , DeletePlanningModal	usePlanningEditView (<code>usePlanningView</code>), usePlanning , useGoBack	app
<code>planning/[id]/edit.vue</code> (<code>/planning/:id/edit</code>)	Éditer un planning existant	PlanningEditor (<code>mode="edit"</code>)	usePlanningEditView	app

Accueil, recherche, paramètres & divers

Page (route)	Objectif	Composants utilisés	Composables/Stores utilisés	Layout
<code>home.vue (/home)</code>	Tableau de bord authentifié : recettes/cookbooks/plannings récents, activité des cookbooks partagés, menu rapide « Nouveau »	AppButton , RecipeCard , PlanningCard , CookbookCard , SharedCookbookCard	useRecipes , usePlanning , useCookbooks , useAuth	app
<code>search.vue (/search)</code>	Recherche globale recettes/plannings/cookbooks, filtres backend + filtre rapide par nom côté client	SearchFilters , RecipeSearchResults , PlanningSearchResults , CookbookSearchResults	useCuisinePreferences , useSearch , useRecipes ; useRecipeStore , usePlanningStore , useCookbookStore	app
<code>import_export.vue (/import_export)</code>	Export/import JSON en masse pour les recettes personnelles et cookbooks possédés	ExportSelectionList , ImportDropzone , ConfirmModal	useApi , usePagination (fetchAllPages), useImportExport	app
<code>settings.vue (/settings)</code>	Paramètres du compte : avatar, nom d'utilisateur, email, mot de passe, liaison Microsoft, préférences de cuisine, suppression de compte	AppButton , AppInput , ConfirmModal , CuisinePreferencesPanel	useAuth , useOAuth , useChangeEmail , useChangePassword , useRecipes ; useUserStore , useToastStore	app
<code>sharedwithme.vue (/sharedwithme)</code>	Lister les cookbooks partagés avec l'utilisateur courant (recherchable)	SharedCookbookCard	useCookbooks (fetchSharedCookbooks), useAuth	app

Dépôt Git

Pour accéder à l'historique des commits/pull effectués, vous pouvez cliquer sur le [lien](#) suivant.
Ci-dessous, vous trouverez l'historique des commits effectués pour ce projet :

Commit	Author	Date	Message
50ecbeb	julien-data-analyst	2026-08-26	Merge pull request #31 from julien-data-analyst/develop
adb8a5f	julien-data-analyst	2026-08-26	Merge pull request #30 from julien-data-analyst/feat(frontend-backend)/update-search-filters
91b8c0f	julien-data-analyst	2026-08-26	fix(frontend): quality code for prettier
3c5de45	julien-data-analyst	2026-08-26	feat(docs): updated api docs
1a3bec2	julien-data-analyst	2026-08-26	feat(frontend): updated filters for taking new attributes and showing new information for planning and cookbooks. Fixed also the dury filters which does now only with the backend.
980288d	julien-data-analyst	2026-08-26	feat(backend): added filters by roles for shared cookbooks
0608af7	julien-data-analyst	2026-08-25	Merge pull request #29 from julien-data-analyst/fix(frontend-backend)/general-fixes
94cdef5	julien-data-analyst	2026-08-25	fix(backend): updated planning/recipe to not being able to change between cookbooks and added THROTTLE management
6cf0c98	julien-data-analyst	2026-08-25	fix(frontend): updated user interface to not import planning into a cookbook
c1c0ca0	julien-data-analyst	2026-08-13	Merge pull request #28 from julien-data-analyst/fix(backend)/icon-OAuth-microsoft
dff4184	julien-data-analyst	2026-08-13	fix(ci): fixed file-higiene check

Commit	Author	Date	Message
798948a	julien-data-analyst	2026-08-13	fix(ci): fixed quality code
b0ddf07	julien-data-analyst	2026-08-13	fix(backend): fixed profile picture for microsoft account
b4ef621	julien-data-analyst	2026-08-09	feat(docs): updated markdown docs to be coherent and User stories
882889c	julien-data-analyst	2026-08-09	fix(prod): fixed pod for using azure client id
f8d072c	julien-data-analyst	2026-08-09	Merge pull request #27 from julien-data-analyst/feat(docker)/add-docker-prod
fe5ff1b	julien-data-analyst	2026-08-09	Merge branch 'develop' into feat(docker)/add-docker-prod
a5539c5	julien-data-analyst	2026-08-09	feat(docker): added prod and fixed versop, for frontend packages.
90ed3a6	julien-data-analyst	2026-08-09	Merge pull request #26 from julien-data-analyst/feat(frontend)/edit-recipes-step-text
07ddcb1	julien-data-analyst	2026-08-09	fix(frontend): fixed quality code for nuxt typecheck
3dce926	julien-data-analyst	2026-08-09	feat(frontend): updated markdown editor to pre-render instantly with tiptap package and added security with URL.
6faff5b	julien-data-analyst	2026-08-09	feat(frontend): added tiptap package for markdown output
e92775e	julien-data-analyst	2026-08-07	Merge pull request #25 from julien-data-analyst/feat(frontend)/general-update-UI-UX
d1f6a62	julien-data-analyst	2026-08-07	fix(frontend): fixed quality code for prettier

Commit	Author	Date	Message
615598f	julien-data-analyst	2026-08-07	feat(frontend): updated bloc filters to select tags/ingredients instead of writting raw text
edd85c0	julien-data-analyst	2026-08-07	fix(backend): fixed quality code for ruff
f34e8e9	julien-data-analyst	2026-08-07	feat(frontend): added culinary preferences for users and used in the search pages
969b8f0	julien-data-analyst	2026-08-07	feat(frontend): updated pagination, notification if deleted, tags capitalization and ingredients proportions for view
7dd87e2	julien-data-analyst	2026-08-07	feat(frontend): updated cookbooks to have the possibility to filters for recipes and plannings.
82e9e91	julien-data-analyst	2026-08-07	fix(backend): updated cookbooks and planning to have into account the favorite attributes
16d6e23	julien-data-analyst	2026-08-07	fix(frontend): forgot to add the typescript for pagination
49dccff	julien-data-analyst	2026-08-07	feat(frontend): added pagination for all main pages
15db0ac	julien-data-analyst	2026-08-07	Merge pull request #24 from julien-data-analyst/feat(frontend)/import-export-recipes-cookbooks
5e3cac8	julien-data-analyst	2026-08-07	feat(frontend): added import/export fonctionnalités for recipes and cookbooks.
b7ee80d	julien-data-analyst	2026-08-07	feat(backend): added possibility to filter by ids for the export and added image in the exported data.
0b87f48	julien-data-analyst	2026-08-07	Merge pull request #23 from julien-data-analyst/feat(frontend)/add-research-filters
cf41de3	julien-data-analyst	2026-08-07	feat(frontend): added filters for dury of two types.

Commit	Author	Date	Message
5480634	julien-data-analyst	2026-08-07	feat(frontend): added search page with different filters (recipes/plannings/cookbooks)
51f45f6	julien-data-analyst	2026-08-07	feat(docs): updated api docs for filters
9d8ce70	julien-data-analyst	2026-08-07	feat(backend): added new filters for recipes to give the name of the cookbook and the planning they were in.
f0bab92	julien-data-analyst	2026-08-07	Merge pull request #22 from julien-data-analyst/feat(frontend)/manage-shared-cookbook
a855f7c	julien-data-analyst	2026-08-07	fix(backend): fixed quality code for basedpyrgjht and ruff
5f32c46	julien-data-analyst	2026-08-06	fix(frontend): fixed quality code for prettier
e096c02	julien-data-analyst	2026-08-06	feat(frontend): update better visualization for messaging and fixed many visuals bugs.
fcd32f8	julien-data-analyst	2026-08-06	feat(backend): added possibility to put messages into planning canal.
9a73909	julien-data-analyst	2026-08-06	feat(frontend): added share fonctionnalités for cookbooks and roles + also added possibilities to communicate (need to fix for the roles).
2d6f918	julien-data-analyst	2026-08-05	Merge pull request #21 from julien-data-analyst/feat(frontend)/maage-cookbooks-personnal
a392f22	julien-data-analyst	2026-08-05	feat(frontend): added possibility to delete or keep recipes/planning when deleting a cookbook.
2173e44	julien-data-analyst	2026-08-05	fix(frontend): fixed quality visual for recipes planning
4e4f88e	julien-data-analyst	2026-08-05	fix(frontend): fixed UI/UX visual to be better for the user.

Commit	Author	Date	Message
2cf75dc	julien-data-analyst	2026-08-05	fix(frontend): fixed quality codes for linters
95f033a	julien-data-analyst	2026-08-05	feat(frontend): added mangaging cookbooks + fixes different fonctionnalities (showing numbers for ingredients, permits to change image and go back to the precedent page)
ef7d32c	julien-data-analyst	2026-08-04	Merge pull request #20 from julien-data-analyst/feat(frontend,backend)/manage-OAuth-already-in-place
28b124c	julien-data-analyst	2026-08-04	fix(cii-cd): fixed quality code for backend and frontend
c39df2a	julien-data-analyst	2026-08-04	feat(frontend): added on frontend the two scenarios with some optimization for OAuth method.
b9df0a7	julien-data-analyst	2026-08-04	fix(backend): fixed email duplicate bugs
959ac9c	julien-data-analyst	2026-08-04	feat(backend): added possibility for the two scenarios to change Oauth into locals and locals into OAuth
f579fec	julien-data-analyst	2026-08-04	Merge pull request #19 from julien-data-analyst/feat(frontend)/manage-planning-recipes
d8b977a	julien-data-analyst	2026-08-04	fix(frontend): fixed quality code for prettier
1e8c69d	julien-data-analyst	2026-08-04	fix(backend): fixed quality code for basedpyright
f953bdb	julien-data-analyst	2026-08-04	fix(backend): fixed quality codes for ruff
7188e7c	julien-data-analyst	2026-08-04	fix(frontend): added message to precise which planning is going to be impacted

Commit	Author	Date	Message
138c4d9	julien-data-analyst	2026-08-04	fix(backend): can now delete a recipe which is already in a planning/cookbook.
7c00bc1	julien-data-analyst	2026-08-04	feat(frontend): first version of managing plannings with possibility to choose between daily / weekly
d1e63c6	julien-data-analyst	2026-08-04	fix(backend): fixed buf unauthorized access on users plannings.
b94ced8	julien-data-analyst	2026-08-04	feat(backend): added attributes for planning with icons and types (journalier/hebdomadaire).
67c3771	julien-data-analyst	2026-08-03	Merge pull request #18 from julien-data-analyst/feat(frontend)/settings-users
3d84d0d	julien-data-analyst	2026-08-03	feat(frontend): added settings fonctionnalités + optimized code for recipes.
a251b12	julien-data-analyst	2026-08-03	feat(backend): added fonctionnalités to implement avatars, change password, email and profiles names but also delete the account.
ba68234	julien-data-analyst	2026-08-03	Merge pull request #17 from julien-data-analyst/feat(frontend)/manage-personnal-recipes
44c67a6	julien-data-analyst	2026-08-03	fix(frontend): fixed quality code with prettier v2
9cb73ab	julien-data-analyst	2026-08-03	fix(frontend): fixed quality code for prettier
5fb5904	julien-data-analyst	2026-08-03	fix(backend): fixed basedpyright error during check
c4473e6	julien-data-analyst	2026-08-03	fix(backend): fixed quality codes for ruff check
999d6a9	julien-data-analyst	2026-08-03	feat(frontend): added prep duty + tags view and source view

Commit	Author	Date	Message
2267208	julien-data-analyst	2026-08-03	feat(frontend): added sidebar, recipes edit and view and added differents components to be reused.
4893ce6	julien-data-analyst	2026-08-03	feat(frontend): added APIs calls for recipes
2d31547	julien-data-analyst	2026-08-03	fix(backend): fixed unauthorized acces recipes from other users
d0d5a88	julien-data-analyst	2026-08-03	feat(frontend): added all web icons
f0e27a5	julien-data-analyst	2026-08-01	Merge pull request #16 from julien-data-analyst/feat(frontend)/add-login-logout-register
0d31d20	julien-data-analyst	2026-07-29	feat(frontend): Django API connected + added login/logout/refresh calls on the web pages and fixed quality code.
a90b86a	julien-data-analyst	2026-07-29	feat(frontend): added zod packages.
e2f6706	julien-data-analyst	2026-07-29	fix(backend): fixed dury tokens and settings for frontend to connect the API
f1ffc22	julien-data-analyst	2026-07-27	feat(frontend): updated public pages with login/logout/public_home/register
569e27f	julien-data-analyst	2026-07-27	feat(frontend): added icons for webapp
2b9c750	julien-data-analyst	2026-07-27	feat(backend): added components for pages
f2d52b8	julien-data-analyst	2026-07-27	fix(frontend): added general template to fix errors
1e88c26	julien-data-analyst	2026-07-27	fix(frontend): fixed tailwinf config

Commit	Author	Date	Message
3fb559a	julien-data-analyst	2026-07-27	Merge pull request #15 from julien-data-analyst/feat(frontend)/init-tailwindcss-chart
000e9af	julien-data-analyst	2026-07-27	feat(frontend): added login/logout/register pages for nuxt
605b530	julien-data-analyst	2026-07-27	feat(frontend): added charts into tailwind config and added principal + speciifc pages to use for completing codes
53f07fd	julien-data-analyst	2026-07-26	Merge pull request #14 from julien-data-analyst/feat(backend)/updated-api-docs-swagger
d6234f4	julien-data-analyst	2026-07-26	Updated README.md docs for backend developed
b238776	julien-data-analyst	2026-07-26	feat(backend): updated docs for APIs/Database/OAuth with also a french version.
976c6c7	julien-data-analyst	2026-07-26	Merge pull request #13 from julien-data-analyst/feat(backend)/add-logout-user
0a32fb8	julien-data-analyst	2026-07-26	feat(backend): added API route for logout
d6c2a09	julien-data-analyst	2026-07-26	Merge pull request #12 from julien-data-analyst/feat(backend)/update-cookbooks-routes
1ec0cc4	julien-data-analyst	2026-07-26	feat(backend): added possibility to add icons for cookbooks/planning and tests for that
7fdd635	julien-data-analyst	2026-07-26	feat(backend): added filters for shared_with_me and to add members from mail
9e858d5	julien-data-analyst	2026-07-26	Merge pull request #11 from julien-data-analyst/feat(backend)/add-comments-cookbooks
9e9b682	julien-data-analyst	2026-07-26	feat(backend): added APIs routes for messaging in a cookbooks + pytest tests + Swagger updated with explaining parameters.

Commit	Author	Date	Message
ac12bf8	julien-data-analyst	2026-07-26	Merge pull request #10 from julien-data-analyst/feat(backend)/import-export-json-cookbooks-recipes
e7466e0	julien-data-analyst	2026-07-26	fix(backend): fixed quality code for import_export tests
38a8800	julien-data-analyst	2026-07-26	fix(backend): updated the latest scripts with a merge into develop
eba4d74	julien-data-analyst	2026-07-26	Merge branch 'develop' into feat(backend)/import-export-json-cookbooks-recipes
f306e25	julien-data-analyst	2026-07-26	feat(backend): added import/export JSON for cookbooks/recipes and tests.
a4cff59	julien-data-analyst	2026-07-26	Merge pull request #8 from julien-data-analyst/feat(backend)/add-filters-get-recipe-cookbook-planning
108a795	julien-data-analyst	2026-07-26	fix(backend): fixed quality code with basedpyright
47d2264	julien-data-analyst	2026-07-26	fix(backend): fixed quality code for filters.py recipes
723c910	julien-data-analyst	2026-07-26	feat(backend): added filters/favorites to recipes/planning/cookbooks with test and explication of the parameters.
1d7f3b2	julien-data-analyst	2026-07-25	Merge pull request #7 from julien-data-analyst/feat(backend)/manage-OAuth-password-users
7881a5f	julien-data-analyst	2026-07-25	feat(backend): added route for changing password
77035a6	julien-data-analyst	2026-07-25	feat(backend): updated oauth docs for Providers and use in the frontend

Commit	Author	Date	Message
aa19f10	julien-data-analyst	2026-07-25	feat(backend): added OAuth Microsoft
68512d8	julien-data-analyst	2026-07-25	Merge pull request #6 from julien-data-analyst/feat(backend)/manage-cookbooks-crud-operation
751d17e	julien-data-analyst	2026-07-25	fix(backend): fixed for update a permission and also quality code with ruff
b1c7c7e	julien-data-analyst	2026-07-25	feat(backend): modified planning codes to fit with the latest updates from develop
71febbb	julien-data-analyst	2026-07-25	Merge branch 'develop' into feat(backend)/manage-cookbooks-crud-operation
70369a7	julien-data-analyst	2026-07-25	feat(backend): added APIs routes for cookbooks CRUD and sharing with tests also
dd61b6b	julien-data-analyst	2026-07-25	Merge pull request #5 from julien-data-analyst/feat(backend)/manage-planning
a55f787	julien-data-analyst	2026-07-25	fix(backend): fix code qualityt for respecting ruff format
58e57f2	julien-data-analyst	2026-07-25	feat(backend): updated docs for changing primary keys for association table planning-recipe
8f257ee	julien-data-analyst	2026-07-25	feat(backend): added tests for planning
286da46	julien-data-analyst	2026-07-25	feat(backend): added APIs routes for managing plannings
c011988	julien-data-analyst	2026-07-25	Merge pull request #4 from julien-data-analyst/feat(backend)/manage-recipe
12868f4	julien-data-analyst	2026-07-25	feat(backend): docs updated for changing some recipes attributes.

Commit	Author	Date	Message
140603c	julien-data-analyst	2026-07-25	feat(backend): added tests for recipes
01b434e	julien-data-analyst	2026-07-25	feat(backend): added APIs routes for recipes
035788a	julien-data-analyst	2026-07-25	Merge pull request #3 from julien-data-analyst/feat(backend)/add-api-user-routes
6bf8931	julien-data-analyst	2026-07-25	feat(backend): added code correction to respect the pre-commit
949c4fe	julien-data-analyst	2026-07-25	feat(backend): updated database documentation and added oauth.md
0e0a79a	julien-data-analyst	2026-07-25	feat(backend): added test for users routes
a53a09e	julien-data-analyst	2026-07-25	feat(backend): added user routes and modify profil icons
53e9570	julien-data-analyst	2026-07-25	Merge pull request #2 from julien-data-analyst/feat(backend)/add-entities-sql-database
29137a3	julien-data-analyst	2026-07-25	feat(backend): added entities and config on the PostgreSQL Database
616eb26	julien-data-analyst	2026-07-22	Merge pull request #1 from julien-data-analyst/feat(workflows)/add-linter-ruff-local-github
e00fef6	julien-data-analyst	2026-07-22	fix(workflow): add prettierignore to ignore the folder .nuxt
d55d8ab	julien-data-analyst	2026-07-22	fix(workflow): fixed node version for using group_by
899c931	julien-data-analyst	2026-07-22	fix(workflow): fix all configuration for linters/ruff/etc to verify code quality
5e11b32	julien-data-analyst	2026-07-20	fix(workflows): fixed level of error for basedpyright.

Commit	Author	Date	Message
6001ce2	julien-data-analyst	2026-07-20	Format Python code with Ruff
4154a38	julien-data-analyst	2026-07-20	Apply pre-commit formatting
d4d928b	julien-data-analyst	2026-07-20	feat(workflows): fixed package nuxt locks
c4530c8	julien-data-analyst	2026-07-20	fix(workflows): fix linters for using uv.lock
c71bf6e	julien-data-analyst	2026-07-20	feat(workflow): added linters for ruff/basedpyright/linter/prettier/typecheck
f42cae0	julien-data-analyst	2026-07-20	feat(frontend): web prototype with AI to have a vision on what web pages we want.
2922721	julien-data-analyst	2026-07-20	feat(docs): init docs with markdown files, User stories and bdd/api schemas.
ea4c2de	julien-data-analyst	2026-07-08	feat(config-env): updated README.md for development purposes.
fd6c1de	julien-data-analyst	2026-07-08	feat(config-env): updated README.md
100364d	julien-data-analyst	2026-07-08	feat(env-config): config of the dev/prod environment with django, nuxt and postgres for services.
b269377	julien-data-analyst	2026-07-07	Initial commit